

Trí Tuệ Nhân Tạo

(Artificial Intelligence)

Lê Thanh Hương

Viện Công nghệ thông tin và Truyền thông
Trường Đại Học Bách Khoa Hà Nội

Nội dung môn học

Chương 1. Tổng quan

Chương 2. Tác tử thông minh

Chương 3. Giải quyết vấn đề

3.1. Tìm kiếm cơ bản

3.2. Tìm kiếm với tri thức bổ sung

3.3. Tìm kiếm dựa trên thỏa mãn ràng buộc

Chương 4. Tri thức và suy diễn

Chương 5. Học máy

Nhắc lại: Tìm kiếm theo cấu trúc cây

```
function TREE-SEARCH(problem, fringe) returns a solution, or failure
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST[problem] applied to STATE(node) succeeds return node
    fringe ← INSERTALL(EXPAND(node, problem), fringe)
```

- Một chiến lược (phương pháp) tìm kiếm = Một cách xác định thứ tự xét các nút của cây

Tìm kiếm với tri thức bổ sung

- Các chiến lược tìm kiếm **cơ bản** (uninformed search strategies) chỉ sử dụng các **thông tin chứa trong định nghĩa** của bài toán
 - Không phù hợp với nhiều bài toán thực tế (do đòi hỏi **chi phí quá cao về thời gian và bộ nhớ**)
- Các chiến lược tìm kiếm **với tri thức bổ sung** (informed search strategies) sử dụng **các tri thức cụ thể của bài toán** → Quá trình tìm kiếm **hiệu quả hơn**
 - Các giải thuật tìm kiếm best-first (**Greedy best-first, A***)
 - Các giải thuật tìm kiếm cục bộ (**Hill-climbing, Simulated annealing, Local beam, Genetic algorithms**)
 - Các giải thuật tìm kiếm đối kháng (**MiniMax, Alpha-beta pruning**)

Best-first search

- **Ý tưởng:** Sử dụng một *hàm đánh giá $f(n)$* cho mỗi nút của cây tìm kiếm
 - Để đánh giá mức độ “*phù hợp*” của nút đó
 - Trong quá trình tìm kiếm, ưu tiên xét các nút có mức độ phù hợp cao nhất
- **Cài đặt giải thuật**
 - Sắp thứ tự các nút trong cấu trúc fringe theo *trật tự giảm dần về mức độ phù hợp*
- **Các trường hợp đặc biệt của giải thuật Best-first search**
 - Greedy best-first search
 - A^* search

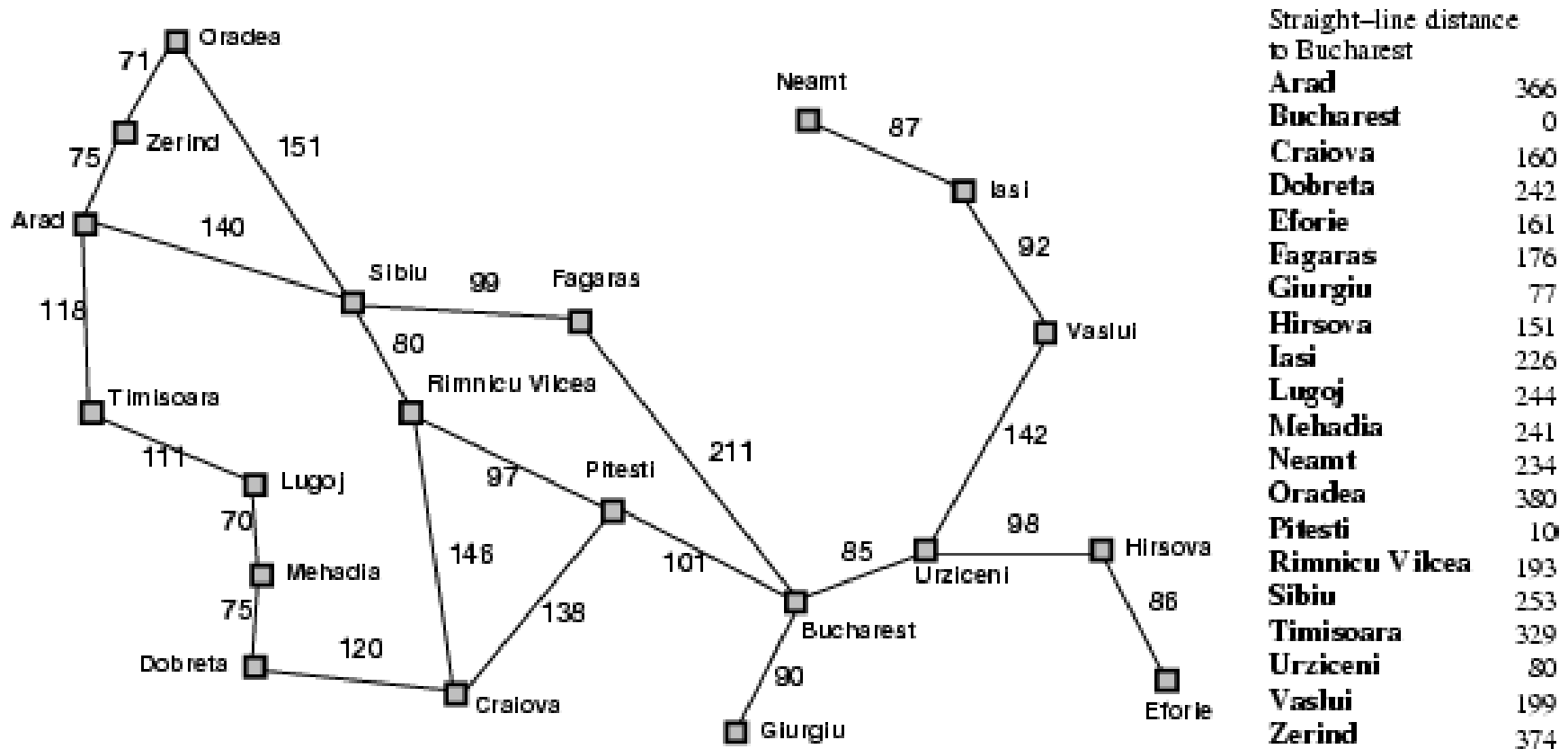
Greedy best-first search

lựa chọn trong các nút biên, nút có k/cách tới đích nhỏ nhất để mở rộng

$f(n) = h(n)$ xây dựng dựa trên thông tin có được từ bài toán, TM 2 đk:

- Hàm đánh giá $f(n)$ là hàm *heuristic* $h(n)$ *$h(n) \geq 0$
 n là đích, $h(n) = 0$*
- Hàm heuristic $h(n)$ đánh giá chi phí để đi từ nút hiện tại n đến nút đích (mục tiêu)
- Ví dụ: Trong bài toán tìm đường đi từ Arad đến Bucharest, sử dụng: $h_{SLD}(n)$ = Ước lượng khoảng cách đường thẳng (“chim bay”) từ thành phố hiện tại n đến Bucharest
- Phương pháp tìm kiếm Greedy best-first search sẽ xét (phát triển) nút “có vẻ” gần với nút đích (mục tiêu) nhất

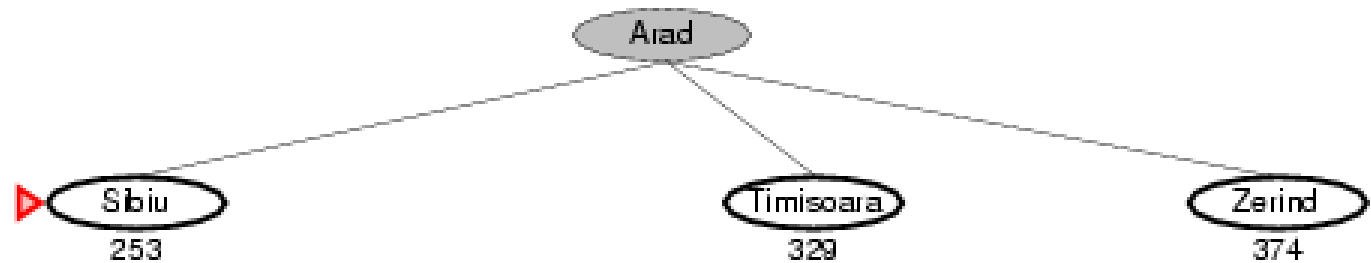
Greedy best-first search – Ví dụ (1)



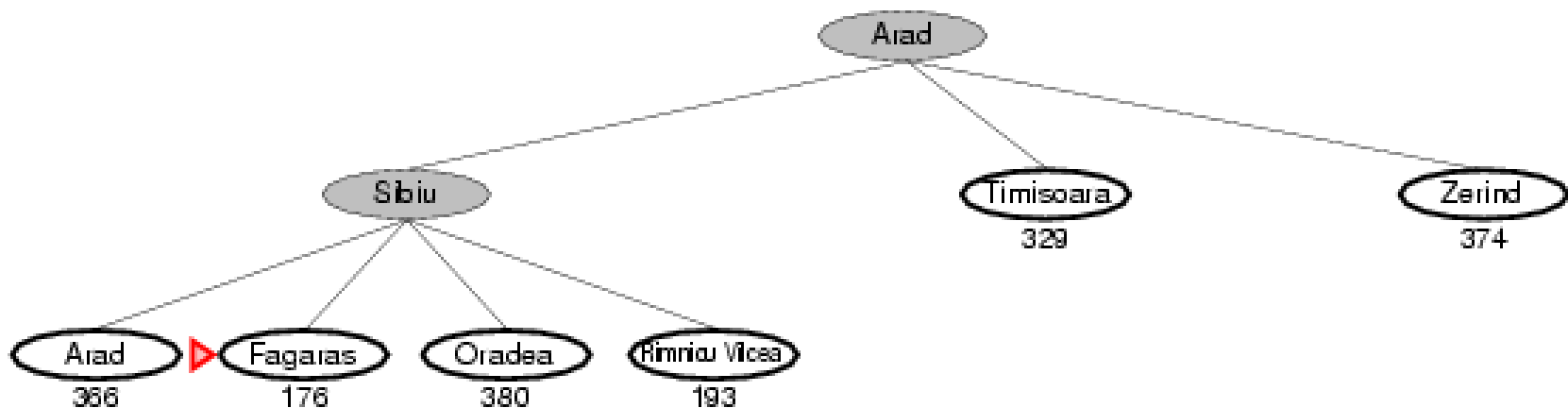
Greedy best-first search – Ví dụ (2)



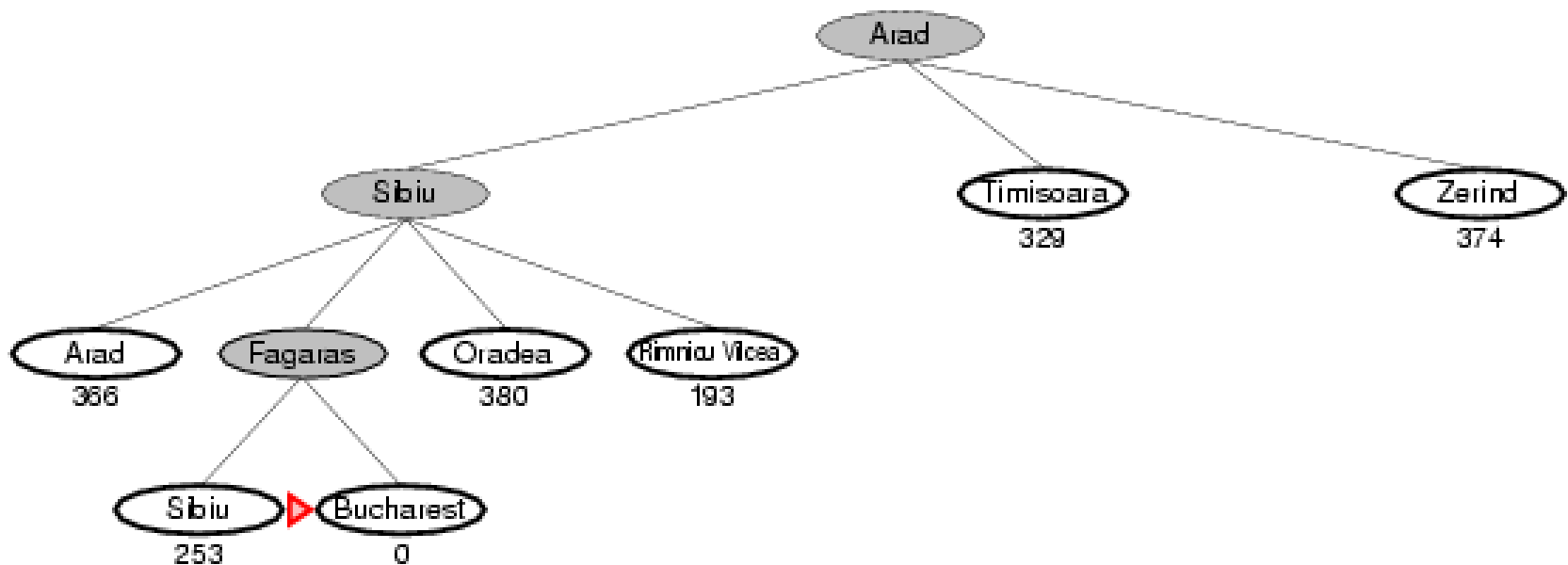
Greedy best-first search – Ví dụ (3)



Greedy best-first search – Ví dụ (4)



Greedy best-first search – Ví dụ (5)



Greedy best-first search – Các đặc điểm

■ Tính hoàn chỉnh?

- Không – Vì có thể vướng (chết tắc) trong các vòng lặp kiểu như:
lasi → Neamt → lasi → Neamt → ... → *lưu ý đưa vào các tình đã có trạng closed*

■ Độ phức tạp về thời gian?

- $O(b^m)$
- Một hàm heuristic tốt có thể mang lại cải thiện lớn

■ Độ phức tạp về bộ nhớ?

- $O(b^m)$ – Lưu giữ tất cả các nút trong bộ nhớ

■ Tính tối ưu?

- Không

*Thuật toán chỉ cố gắng tìm tốt nhất tại 1 bước
mà ko tính tới tổ hợp các bước nên đường đi
tìm thấy có thể ko phải là đn*

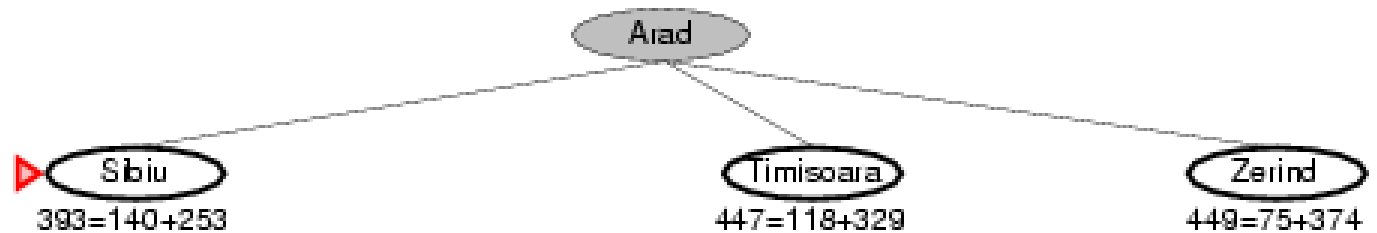
A* search

- **Ý tưởng:** Tránh việc xét (phát triển) các nhánh tìm kiếm đã xác định (cho đến thời điểm hiện tại) là có **chi phí cao**
- Sử dụng hàm đánh giá $f(n) = g(n) + h(n)$
 - $g(n) =$ **chi phí từ nút gốc cho đến nút hiện tại n**
 - $h(n) =$ **chi phí ước lượng từ nút hiện tại n tới đích**
 - $f(n) =$ **chi phí tổng thể ước lượng** của đường đi qua nút hiện tại n đến đích

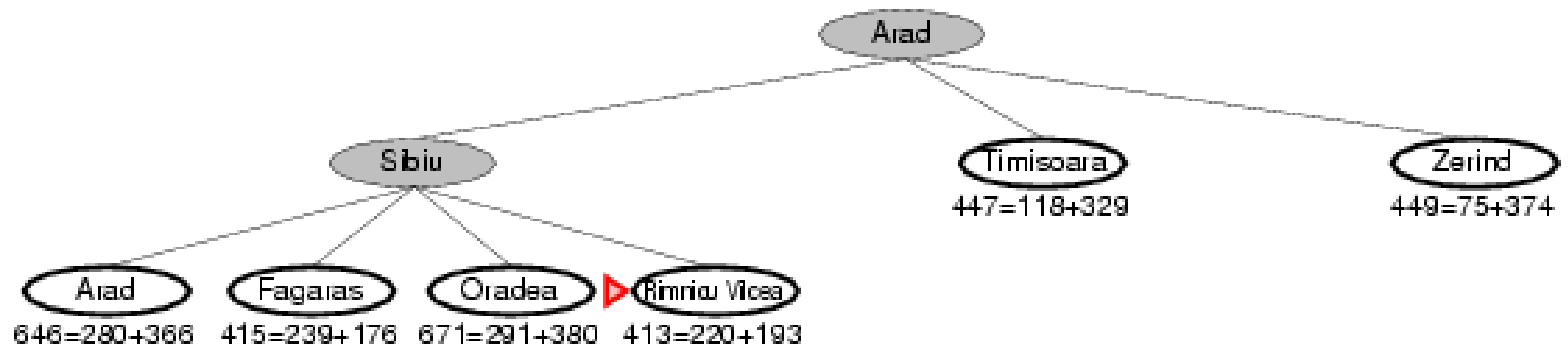
A* search – Ví dụ (1)



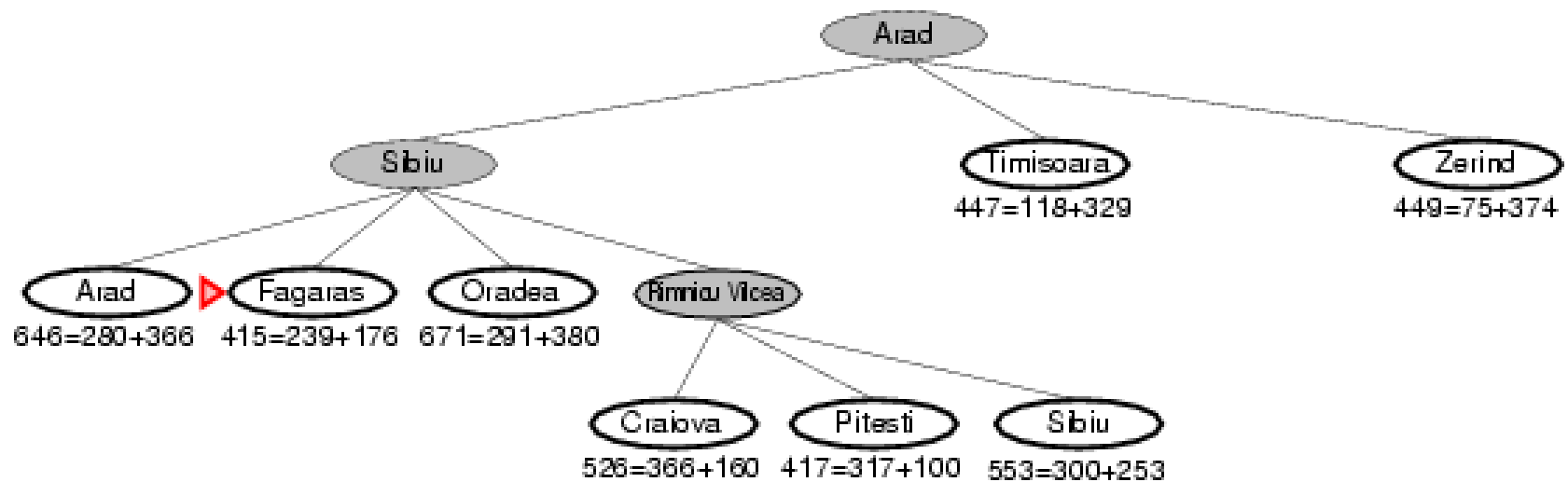
A* search – Ví dụ (2)



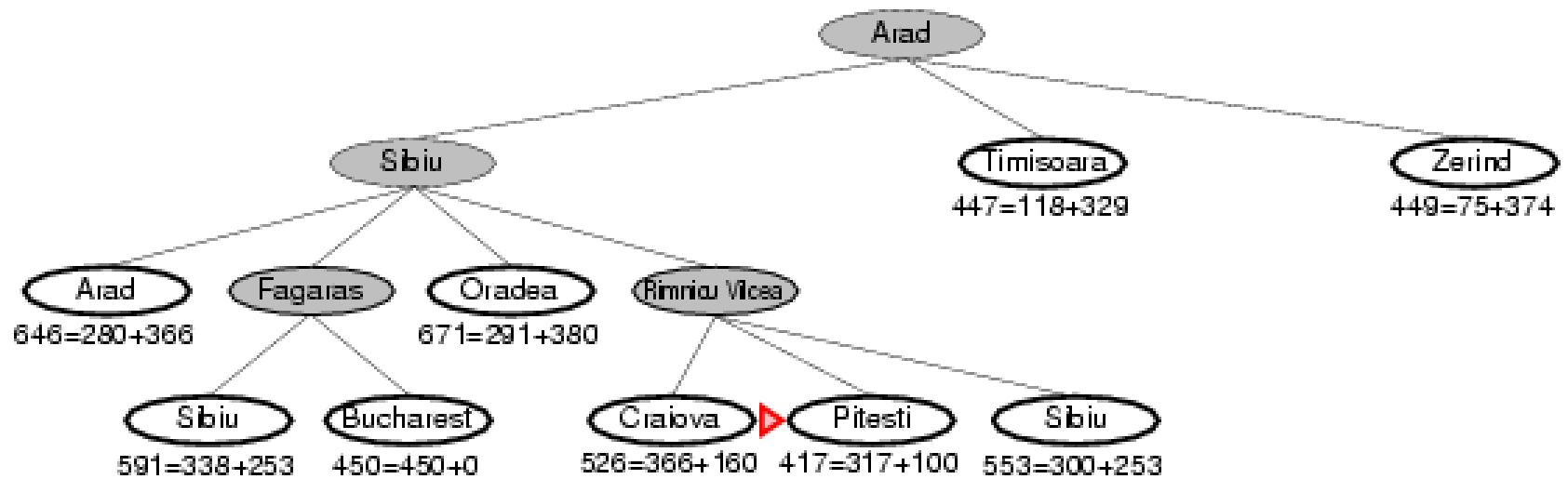
A* search – Ví dụ (3)



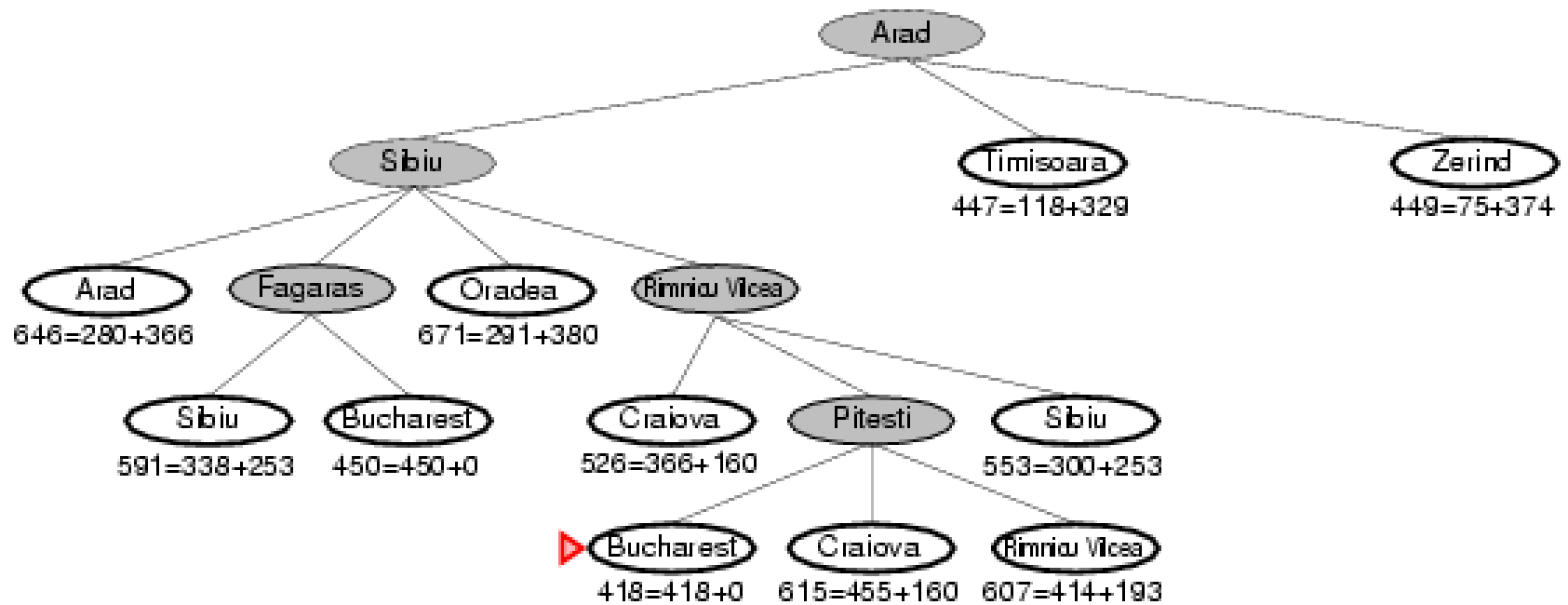
A* search – Ví dụ (4)

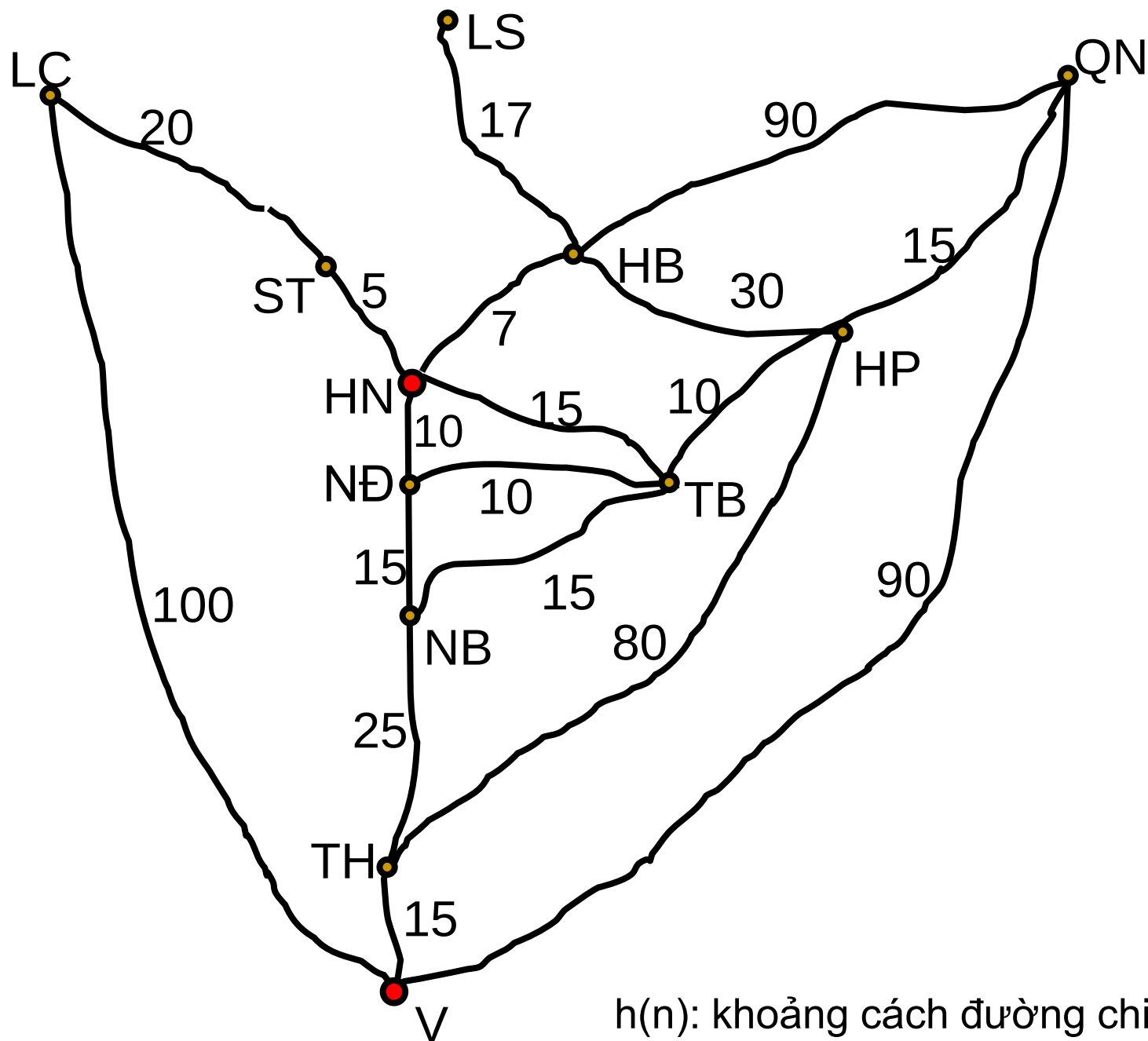


A* search – Ví dụ (5)



A* search – Ví dụ (6)





n	h(n)
HN	50
ST	60
LC	75
HB	65
LS	70
HP	80
QN	80
TB	55
NĐ	45
NB	20
TH	15
V	0

Greedy best-first search

n	successors (n)	fringe	closed
HN50	ST, HB, TB, ND	HN50	
ND45	HN, TB, NB	ST60, HB65, TB55, ND45	HN
NB20	ND, TB, TH	ST60, HB65, TB55, TB55	HN, ND
TH15	NB, HP, V	NB20	
		ST60, HB65, TB55, TB55	HN, ND, NB
		ND45, TB55 , TH15	
		ST60, HB65, TB55, TB55	HN, ND, NB, TH
		ND45, TB55 , NB20, HP80,	
		V0	
		ST60, HB65, TB55, TB55	HN, ND, NB, TH, V
		ND45, TB55 , NB20, HP80,	
		TH15, LC75, QN80	

A* search : $f(n) = g(n) + h(n)$

n	successors (n)	fringe	closed
HN	ST5, HB7, TB15, NØ10	<u>HN50</u>	HN
NØ10	HN10, TB10, NB15	ST65, HB72, TB70, <u>NØ55</u> ST65, HB72, TB70, <u>NB45</u>	HN, NØ
NB25	NØ15, TB15, TH25	ST65, HB72, TB70, <u>TH65</u>	HN, NØ, NB
ST5	HN5, LC20	HB72, TB70, TH65, LC100	HN, NØ, NB, ST
TH50	NB25, HP80, V15	HB72, TB70, LC100, HP210	HN, NØ, NB, ST, TH
V65	LC100, TH15, QN90	<u>V65</u> HB72, TB70, LC100, HP210, QN235	HN, NØ, NB, ST, TH, V

HN → NØ → NB → TH → V



n	successor(n)	fringe	closed
A	B 2, C 1, D 2	B 7, C 6, D 8	A
C	G 8, H 9	B 7, <u>D 8</u> , G 10, H 9	AC
B	E 10, F 10	D 8, G 10, H 9, E 11, F 10	ACBD
D	I 7, J 10	G 10, H 9, E 11, F 10, I 10, J 10	
H			

A* search: các đặc điểm

⊗ Tính hoàn chỉnh?

ko mở rộ các trạng thái
↑ lưu trữ closed

- Nếu không gian các trạng thái là hữu hạn và có giải pháp để tránh việc xét (lặp) lại các trạng thái, thì giải thuật A* là hoàn chỉnh (tìm được lời giải) – nhưng không đảm bảo là tối ưu
- Nếu không gian các trạng thái là hữu hạn và không có giải pháp để tránh việc xét (lặp) lại các trạng thái, thì giải thuật A* là không hoàn chỉnh
- Nếu không gian các trạng thái là vô hạn, thì giải thuật A* là không hoàn chỉnh
- Khi nào thì A* tối ưu?

Các ước lượng chấp nhận được

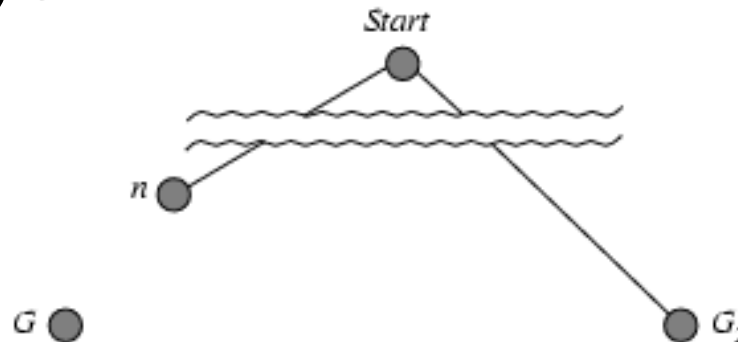
- Một ước lượng $h(n)$ được xem là chấp nhận được nếu đối với mọi nút n : $0 \leq h(n) \leq h^*(n)$, trong đó $h^*(n)$ là chi phí thật (thực tế) để đi từ nút n đến đích
- Một ước lượng chấp nhận được không bao giờ đánh giá quá cao (overestimate) đối với chi phí để đi tới đích
 - Thực chất, ước lượng chấp nhận được có xu hướng đánh giá “lạc quan”
- Ví dụ: Ước lượng $h_{SLD}(n)$ đánh giá thấp hơn khoảng cách đường đi thực tế
- **Định lý:** Nếu $h(n)$ là đánh giá chấp nhận được, thì phương pháp tìm kiếm A^* sử dụng giải thuật TREE-SEARCH là tối ưu

Tính tối ưu của A^* - Chứng minh (1)

Xét ra G_2 trước G

- Giả sử có một đích không tối ưu (suboptimal goal) G_2 được sinh ra và lưu trong cấu trúc *fringe*. Gọi n là một nút chưa xét trong cấu trúc *fringe* sao cho n nằm trên một đường đi ngắn nhất đến một đích tối ưu (optimal goal) G

Lấy ra n khi f là min



$$\begin{aligned}h(G) &= 0 \\h(G_2) &= 0 \\g(G) &< g(G_2) \\f(x) &= g(x) + h(x) \\h(x) &\leq h^*(x)\end{aligned}$$

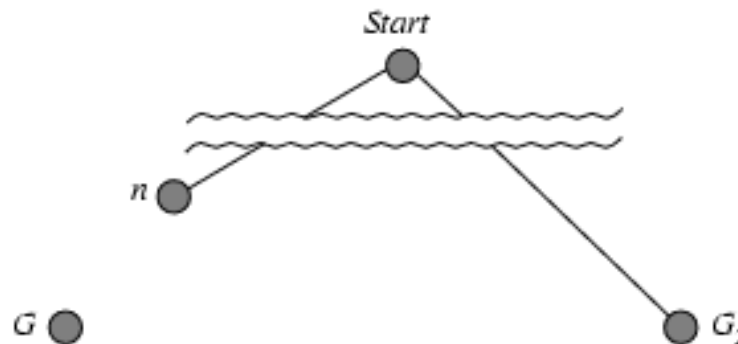
- Ta có: 1) $f(G_2) = g(G_2)$
- Ta có: 2) $g(G_2) > g(G)$
- Ta có: 3) $f(G) = g(G)$
- Từ 1)+2)+3) suy ra: 4) $f(G_2) > f(G)$

vì $h(G_2) = 0$

vì G_2 là đích không tối ưu

vì $h(G) = 0$

Tính tối ưu của A^* - Chứng minh (2)



$f(n) \succ f(G_2) (*)$
 $f(G_2) > f(G) (**)$
 $f(n) \leq f(G) (***)$
 $\Rightarrow$ Mâu thuẫn

- Ta có: 5) $h(n) \leq h^*(n)$ vì h là ước lượng chấp nhận được
- Từ 5) suy ra: 6) $g(n) + h(n) \leq g(n) + h^*(n)$
- Ta có: 7) $g(n) + h^*(n) = f(G)$ vì n nằm trên đường đi tới G
- Từ 6)+7) suy ra: 8) $f(n) \leq f(G)$
- Từ 4)+8) suy ra: $f(G_2) > f(n)$. Tức là, giải thuật A^* không bao giờ xét G_2

Các ước lượng chấp nhận được (1)

Ví dụ đối với trò chơi ô chữ 8 số:

- $h_1(n)$ = số các ô chữ nằm ở sai vị trí (so với vị trí của ô chữ đầy ở trạng thái đích)
- $h_2(n)$ = khoảng cách dịch chuyển ($\leftarrow, \rightarrow, \uparrow, \downarrow$) ngắn nhất để dịch chuyển các ô chữ nằm sai vị trí về vị trí đúng

- $h_1(S) = ?$

- $h_2(S) = ?$

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Các ước lượng chấp nhận được (2)

Ví dụ đối với trò chơi ô chữ 8 số:

- $h_1(n)$ = số các ô chữ nằm ở sai vị trí (so với vị trí của ô chữ đầy ở trạng thái đích)
- $h_2(n)$ = khoảng cách dịch chuyển ($\leftarrow, \rightarrow, \uparrow, \downarrow$) ngắn nhất để dịch chuyển các ô chữ nằm sai vị trí về vị trí đúng

- $h_1(S) = 8$

- $h_2(S) = 3+1+2+2+2+3+3+2 = 18$

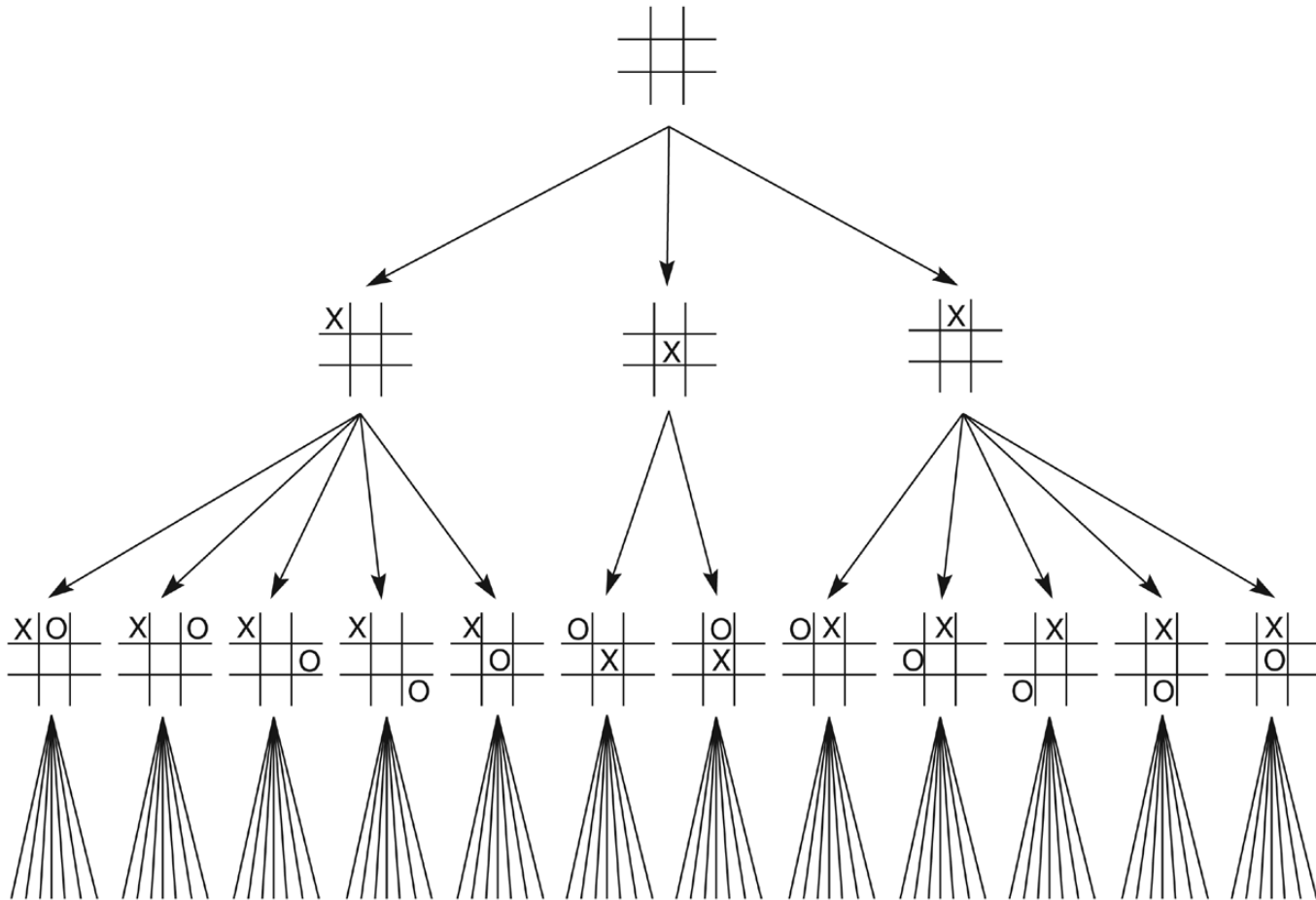
7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

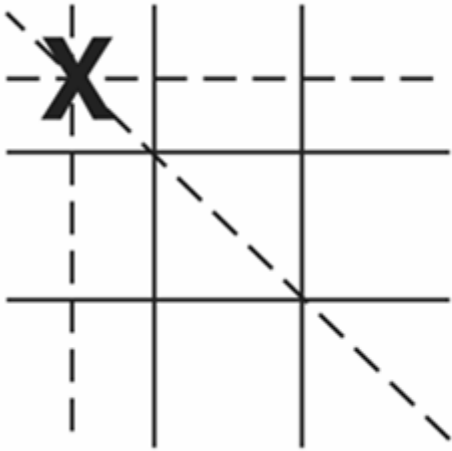
Goal State

Trò chơi Tic-tac-toe

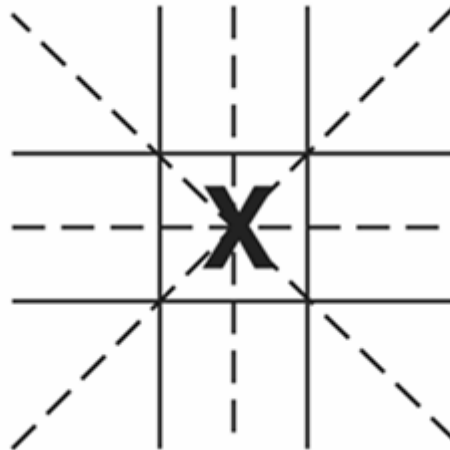


KGTT của tic-tac-toe được thu nhỏ nhờ tính đối xứng của các trạng thái

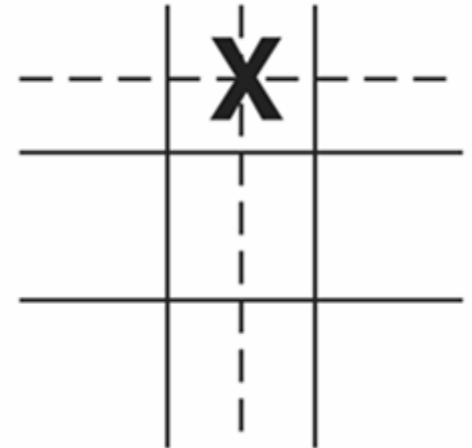
Phép đo heuristic



Chiếm 3 đường



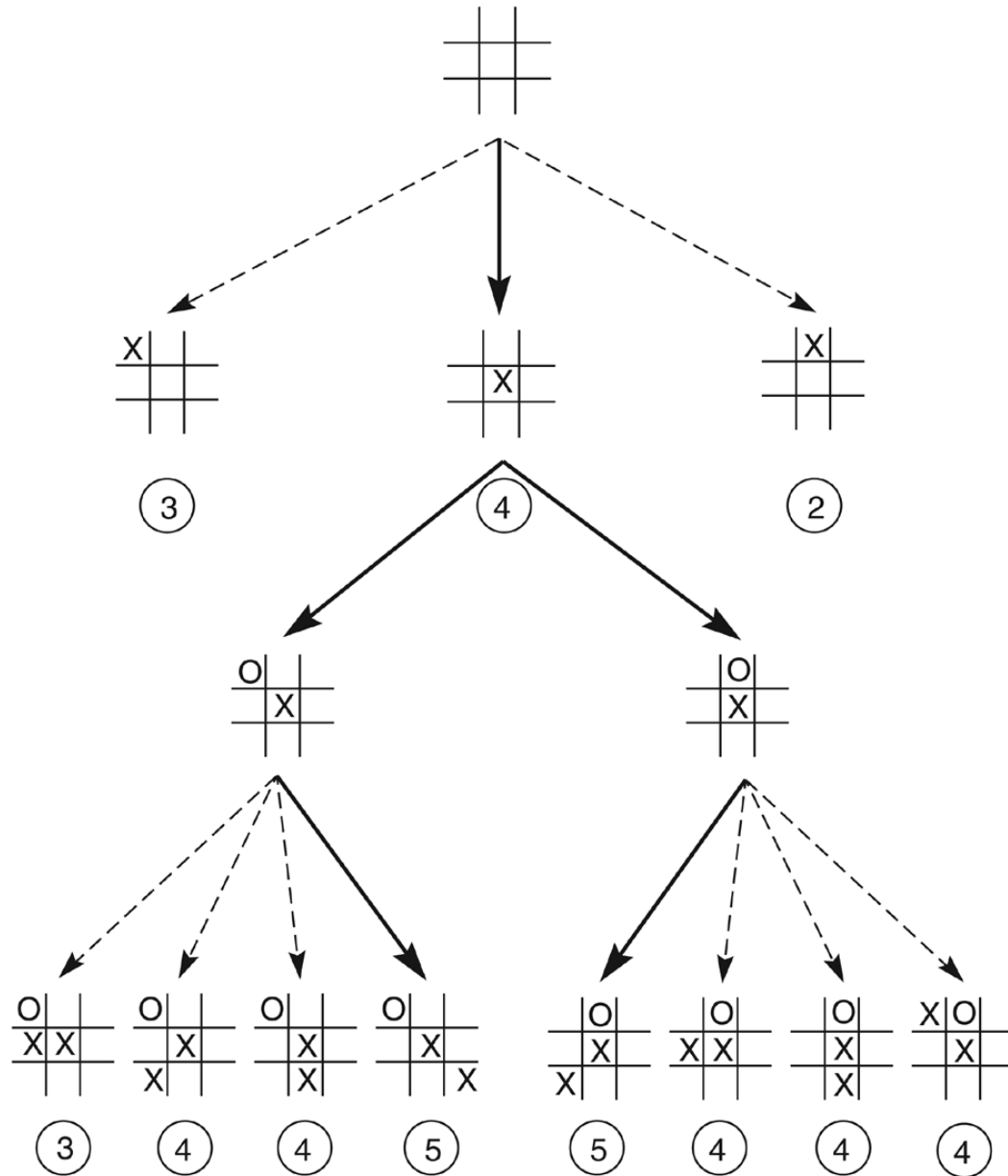
Chiếm 4 đường



Chiếm 2 đường

Heuristic “*Số đường thắng nhiều nhất*” áp dụng cho các nút con đầu tiên trong tic-tac-toe.

Phép đo heuristic



Ước lượng ưu thế

- Ước lượng h_2 được gọi là **ưu thế hơn / trội hơn** (dominate) ước lượng h_1 nếu:
 - $h^*(n) \geq h_2(n) \geq h_1(n)$ đối với tất cả các nút n
- Nếu ước lượng h_2 ưu thế hơn ước lượng h_1 , thì h_2 tốt hơn (nên được sử dụng hơn) cho quá trình tìm kiếm
- Trong ví dụ (ô chữ 8 số) ở trên: Chi phí tìm kiếm = Số lượng trung bình của các nút phải xét:
 - Với độ sâu $d=12$
 - IDS (Tìm kiếm sâu dần): 3.644.035 nút phải xét
 - A*(sử dụng ước lượng h_1): 227 nút phải xét
 - A*(sử dụng ước lượng h_2): 73 nút phải xét
 - Với độ sâu $d=24$
 - IDS (Tìm kiếm sâu dần): Quá nhiều nút phải xét
 - A*(sử dụng ước lượng h_1): 39.135 nút phải xét
 - A*(sử dụng ước lượng h_2): 1.641 nút phải xét

Các ước lượng kiên định

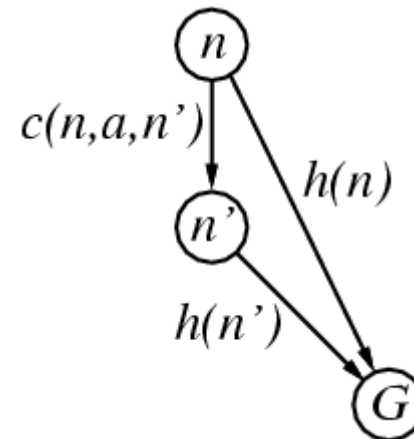
- Một ước lượng h được xem là **kiên định (consistent)**, nếu với mọi nút n và với mọi nút tiếp theo n' của n (được sinh ra bởi hành động a):

$$h(n) \leq c(n, a, n') + h(n')$$

chi phí thực tế

- Nếu ước lượng h là kiên định, ta có:

$$\begin{aligned} f(n') &= g(n') + h(n') \\ &= g(n) + c(n, a, n') + h(n') \\ &\geq g(n) + h(n) = f(n) \end{aligned}$$



Nghĩa là: **$f(n)$ không giảm** trong bất kỳ đường đi (tìm kiếm) nào đi qua n

- Định lý:** Nếu **$h(n)$ là kiên định**, thì **phương pháp tìm kiếm A*** sử dụng giải thuật **GRAPH-SEARCH** là **tối ưu**

Các đặc điểm của A^*

■ Tính hoàn chỉnh?

- Có (trừ khi có rất nhiều các nút có chi phí $f \leq f(G)^{\uparrow}$)

nếu không gian trạng thái là vô hạn

■ Độ phức tạp về thời gian?

- **Bậc của hàm mũ** – Số lượng các nút được xét là hàm mũ của độ dài đường đi của lời giải

■ Độ phức tạp về bộ nhớ?

- **Lưu giữ tất cả các nút trong bộ nhớ**

■ Tính tối ưu?

- Có (đối với điều kiện đặc biệt)

A* vs. UCS

^{thực}

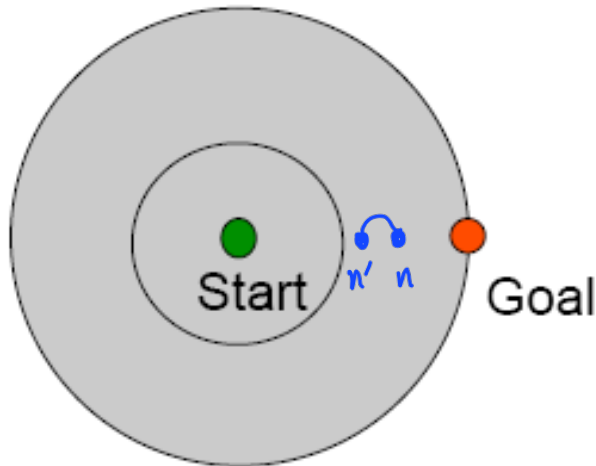
$c(n' \rightarrow n)$

chi phí giữa 2 đỉnh kế nhau

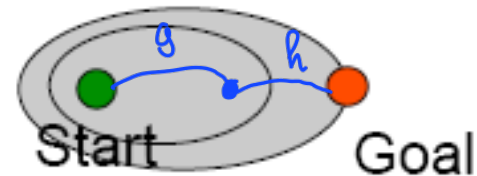
^{thực}

$f(n) = g(n) + h(n)$

- Tìm kiếm với chi phí cực tiểu (UCS) phát triển theo mọi hướng



- Tìm kiếm A* phát triển chủ yếu theo hướng tới đích, nhưng đảm bảo tính tối ưu



Các thuật toán tìm kiếm ở trên đều dựa trên việc khảo sát không gian tìm kiếm một cách hệ thống, tức là thực hiện việc xem xét 1 cách tuần tự các trạng thái để tìm đường đi tối ưu đến đích, do vậy không thích hợp với bài toán có không gian trạng thái rất lớn.

Bên cạnh đó, trong nhiều bài toán tối ưu, các điều kiện thường phức tạp, khó tìm được lời giải bằng sử dụng đường đi để khảo sát không gian tìm kiếm (như các thuật toán đã xem xét) :

- Tìm trạng thái đích (không phải tìm các bước, hành động đến đích)
- Bài toán tối ưu có nhiều cực trị địa phương

Các giải thuật tìm kiếm cục bộ

- Có thể sử dụng cho những bài toán kể trên
- Chỉ lưu một trạng thái, không lưu hành động dẫn đến trạng thái

Mục tiêu: cải thiện trạng thái hiện thời theo 1 tiêu chí nào đó

- Thuật toán 8 quân hậu: tìm vị trí các quân mà không quan tâm tới đường đi dẫn tới trạng thái đó

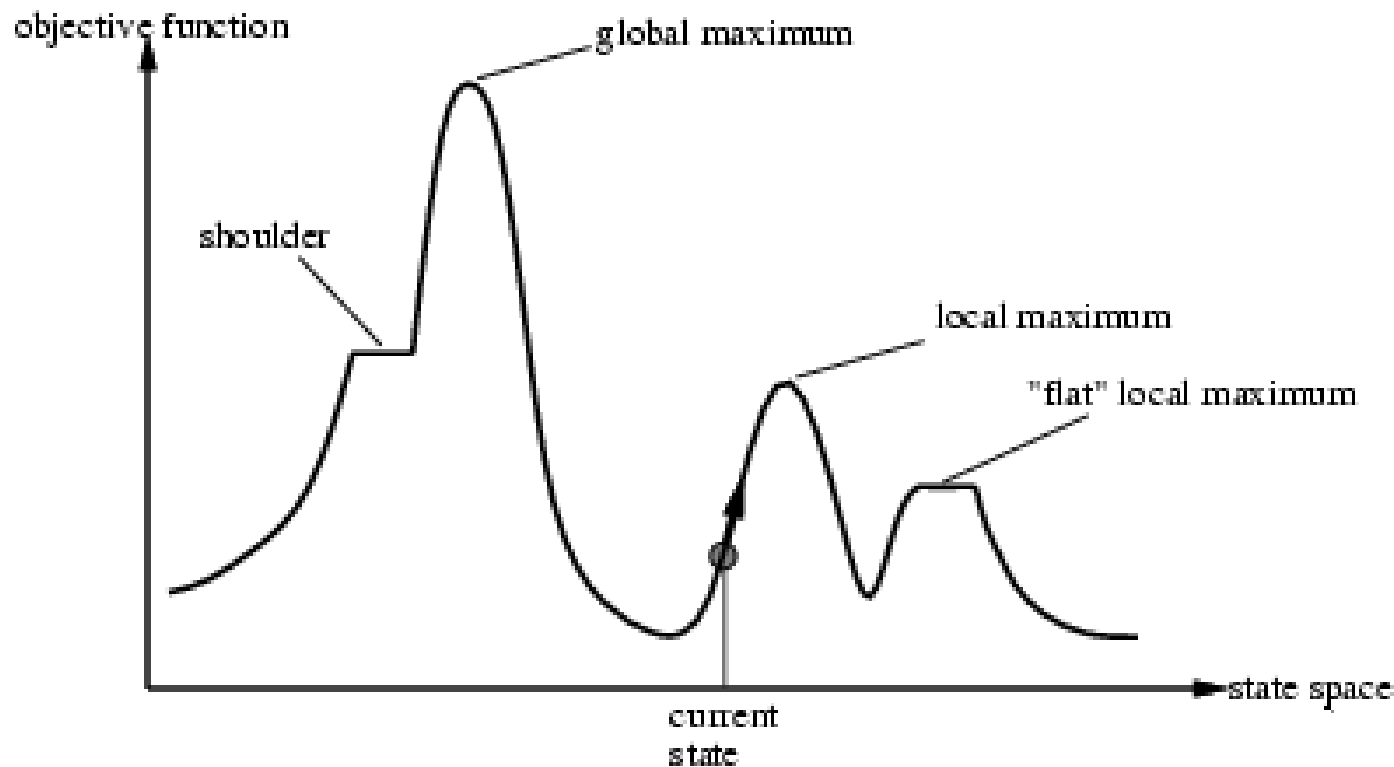
Các giải thuật tìm kiếm cục bộ

- Trong nhiều bài toán tối ưu, các đặc điểm thường phức tạp và ta không thể tìm được lời giải tối ưu.
 - Trạng thái đích = Lời giải của bài toán phải thoả mãn ràng buộc nào đó.
 - Ví dụ: Bài toán n quân hậu (bố trí n quân hậu trên một bàn cờ kích thước $n \times n$, sao cho các quân hậu không ăn nhau)
 - Bài toán tối ưu có nhiều điểm cực trị địa phương.
- Trong những bài toán như thế, chúng ta có thể sử dụng các giải thuật tìm kiếm cục bộ
- Tại mỗi thời điểm, chỉ lưu một trạng thái “hiện thời” duy nhất. Mục tiêu: cố gắng “cải thiện” trạng thái (cấu hình) hiện thời này đối với một tiêu chí nào đó (định trước)

Tìm kiếm leo đồi: minh họa

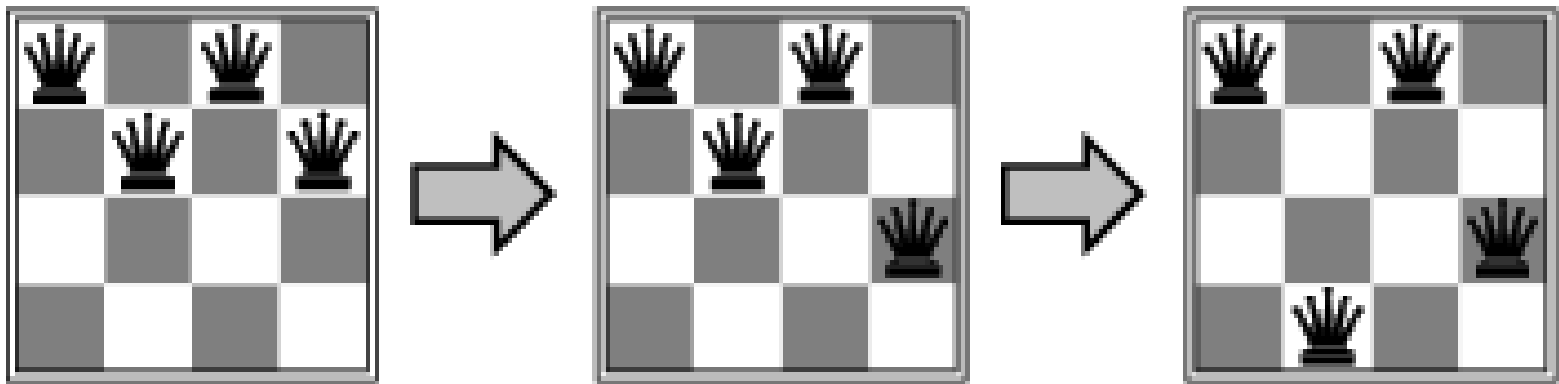
Cơ chế: tạo ra hàng xóm cho trạng thái hiện thời và di chuyển sang hàng xóm có hàm mục tiêu tốt hơn

- Nhược điểm: Tùy vào trạng thái đầu, giải thuật tìm kiếm leo đồi có thể “tắc” ở các điểm cực đại cục bộ (local maxima)
 - Không tìm được lời giải tối ưu toàn cục (global optimal solution)



Ví dụ: Bài toán n quân hậu

- Bố trí n ($=4$) quân hậu trên một bàn cờ có kích thước $n \times n$, sao cho không có 2 quân hậu nào trên cùng hàng, hoặc trên cùng cột, hoặc trên cùng đường chéo

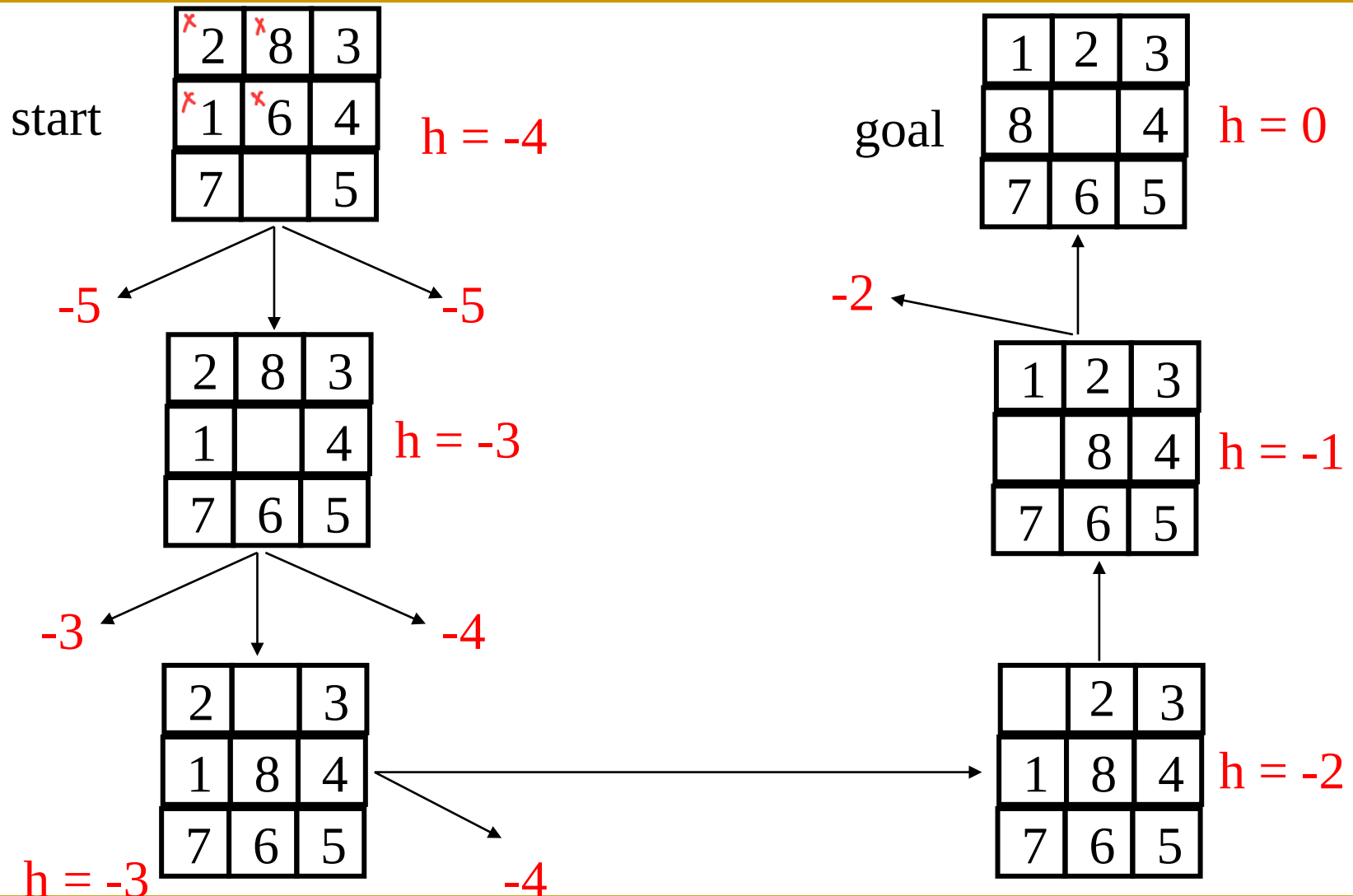


Tìm kiếm leo đồi: giải thuật

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
  inputs: problem, a problem
  local variables: current, a node
                     neighbor, a node

  current ← MAKE-NODE(INITIAL-STATE[problem])
  loop do
    neighbor ← a highest-valued successor of current
    if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
    current ← neighbor
```

Tìm kiếm leo đồi: bài toán ô chữ

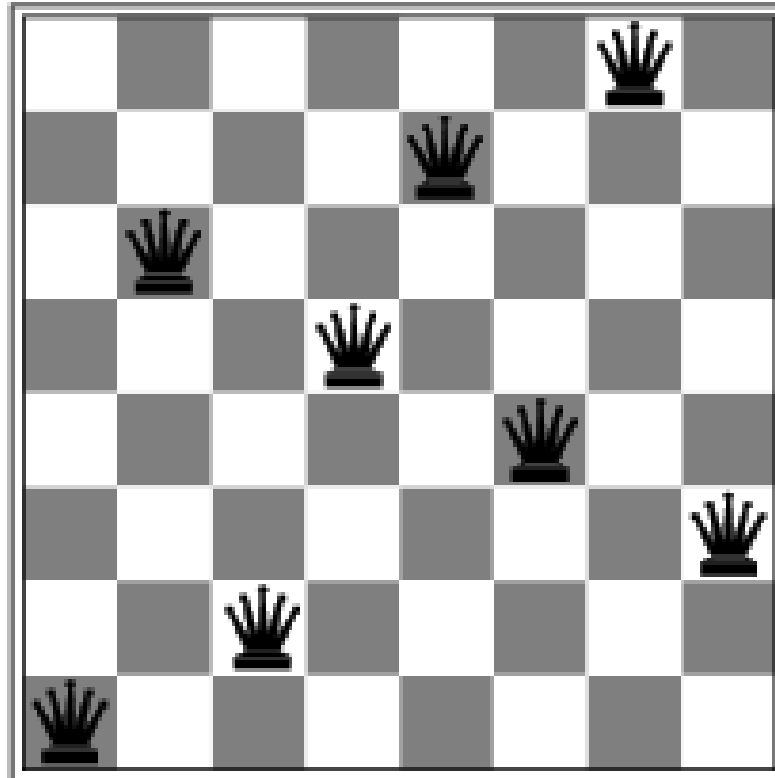


Tìm kiếm leo đồi: bài toán 8 quân hậu (1)

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14		13	16	13	16
	14	17	15		14	16	16
17		16	18	15		15	
18	14		15	15	14		16
14	14	13	17	12	14	12	18

- Ước lượng h = tổng số các cặp quân hậu ăn nhau, hoặc là trực tiếp hoặc gián tiếp
- Trong trạng thái (bàn cờ) trên: $h = 17$

Tìm kiếm leo đồi: bài toán 8 quân hậu (2)



- Thuật toán ko có khả năng đi xuống nên ko thoát khỏi được các cực trị địa phương khi rơi vào
- Thuật toán ko có tính hoàn chỉnh
 - + ko đi tiếp được
 - + Những k/quả chưa p' là trạng thái ko có cặp nào ăn nhau

- Trạng thái bàn cờ trên là một giải pháp tối ưu cục bộ (a local minimum)
 - Với ước lượng $h = 1$ (vẫn còn 1 cặp hậu ăn nhau)

Simulated annealing search

- Dựa trên quá trình tôi ủ (annealing process): Kim loại nguội đi và lạnh cứng lại thành cấu trúc kết tinh
- Phương pháp tìm kiếm Simulated Annealing có thể tránh được các điểm tối ưu cục bộ (local optima)
- Phương pháp tìm kiếm Simulated Annealing sử dụng chiến lược tìm kiếm ngẫu nhiên, trong đó chấp nhận các thay đổi làm tăng giá trị hàm mục tiêu (i.e., cần tối ưu) và *cũng chấp nhận (có hạn chế) các thay đổi làm giảm*
- Phương pháp tìm kiếm Simulated Annealing sử dụng một tham số điều khiển T (như trong các hệ thống nhiệt độ)
 - Bắt đầu thì T nhận giá trị cao, và giảm dần về 0

Simulated annealing search: giải thuật

- Ý tưởng: Thoát khỏi (vượt qua) các điểm tối ưu cục bộ bằng cách cho phép cả các dịch chuyển “tồi” từ trạng thái hiện thời, nhưng giảm dần tần xuất của các di chuyển tồi này

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
           schedule, a mapping from time to “temperature”
  local variables: current, a node
                   next, a node
                   T, a “temperature” controlling prob. of downward steps

  current ← MAKE-NODE(INITIAL-STATE[problem])
  for t ← 1 to ∞ do
    T ← schedule[t]
    if T = 0 then return current
    next ← a randomly selected successor of current
     $\Delta E \leftarrow \text{VALUE}[\textit{next}] - \text{VALUE}[\textit{current}]$ 
    if  $\Delta E > 0$  then current ← next
    else current ← next only with probability  $e^{\Delta E/T}$ 
```

Trí tuệ nhân tạo

cho phép đi đến các trạng thái tồi hơn ở giai đoạn đầu và giảm dần tần suất các di chuyển ấy ở giai đoạn sau

Simulated annealing search: các đặc điểm

- (Có thể chứng minh được) Nếu giá trị của tham số T (xác định mức độ giảm tần xuất đối với các di chuyển tồi) giảm chậm, thì phương pháp tìm kiếm Simulated Annealing sẽ tìm được lời giải tối ưu toàn cục với xác suất xấp xỉ 1
- Phương pháp tìm kiếm Simulated Annealing Search rất hay được sử dụng trong các lĩnh vực: thiết kế sơ đồ bảng mạch VLSI, lập lịch bay, ...

Local beam search

- Ở mỗi thời điểm (trong quá trình tìm kiếm), luôn lưu giữ k – thay vì chỉ 1 – trạng thái tốt nhất
k qtr` tìm kiếm cục bộ độc lập với nhau → tỷ xác suất tìm đc lời giải tối ưu
- Bắt đầu giải thuật: Chọn k trạng thái ngẫu nhiên
- Ở mỗi bước tìm kiếm, sinh ra tất cả các trạng thái kế tiếp của k trạng thái này
- Nếu một trong số các trạng thái là trạng thái đích, thì giải thuật kết thúc (thành công); nếu không, thì chọn k trạng thái tiếp theo tốt nhất (từ tập các trạng thái tiếp theo), và lặp lại bước trên

Giải thuật di truyền

- Dựa trên sự tương tự với quá trình chọn lọc tự nhiên và thuyết tiến hoá của Đac-uyn
- Ý tưởng học tập từ quá trình sinh tồn và chọn lọc tự nhiên của sinh vật, trong đó: Cá thể với khả năng thích nghi cao hơn sẽ chiếm ưu thế và tồn tại

abc

Giải thuật di truyền: giới thiệu

- Dựa trên (bắt chước) quá trình tiến hóa tự nhiên trong sinh học
- Áp dụng phương pháp tìm kiếm ngẫu nhiên (stochastic search) để tìm được lời giải (vd: một hàm mục tiêu, một mô hình phân lớp, ...) tối ưu | *thể hệ tiếp theo sẽ được sinh ra ngẫu nhiên*
- Giải thuật di truyền (Genetic Algorithm – GA) có khả năng tìm được các lời giải tốt thậm chí ngay cả với các không gian tìm kiếm (lời giải) không liên tục rất phức tạp
- Mỗi khả năng của lời giải được biểu diễn bằng một chuỗi nhị phân (vd: 100101101) – được gọi là **nhễm sắc thể (chromosome)**
 - Việc biểu diễn này phụ thuộc vào từng bài toán cụ thể*chuỗi bộ gen của 1 sinh vật + quy định sự hình thành của protein có vai trò quyết định trong di truyền*

Giải thuật di truyền: mô tả

- Xây dựng (khởi tạo) **quần thể (population) ban đầu**
 - Tạo nên một số các giả thiết (khả năng của lời giải) ban đầu
 - Mỗi giả thiết khác các giả thiết khác (vd: khác nhau đối với các giá trị của một số tham số nào đó của bài toán)
- Đánh giá quần thể
 - Đánh giá (cho điểm) mỗi giả thiết (vd: bằng cách kiểm tra độ chính xác của hệ thống trên một tập dữ liệu kiểm thử)
 - Trong lĩnh vực sinh học, điểm đánh giá này của mỗi giả thiết được gọi là **độ phù hợp (fitness)** của giả thiết đó
 - Xếp hạng các giả thiết theo mức độ phù hợp của chúng, và chỉ giữ lại các giả thiết tốt nhất (gọi là **các giả thiết phù hợp nhất – survival of the fittest**)
- Sản sinh ra **thế hệ tiếp theo (next generation)**
 - Thay đổi ngẫu nhiên các giả thiết để sản sinh ra thế hệ tiếp theo (gọi là **các con cháu – offspring**)
- Lặp lại quá trình trên cho đến khi ở một thế hệ nào đó có giả thiết tốt nhất có độ phù hợp cao hơn giá trị phù hợp mong muốn (định trước)

GA(Fitness, θ , n , r_{co} , r_{mu})

đánh giá sự phù hợp
ngưỡng yêu cầu tối thiểu của sự phù hợp
Fitness: A function that produces the score (fitness) given a hypothesis

θ : The desired fitness value (i.e., a threshold specifying the termination condition)

số lượng cá thể trong quần thể
tỉ lệ lai ghép
 n : The number of hypotheses in the population

r_{co} : The percentage of the population influenced by the *crossover* operator at each step

tỉ lệ đột biến
 r_{mu} : The percentage of the population influenced by the *mutation* operator at each step

Initialize the population: $H \leftarrow$ Randomly generate n hypotheses

Evaluate the initial population. For each $h \in H$: compute Fitness(h)

while ($\max_{\{h \in H\}} \text{Fitness}(h) < \theta$) do

$H^{\text{next}} \leftarrow \emptyset$

Reproduction (Replication). Probabilistically select $(1 - r_{co}) \cdot n$ hypotheses of H to add to H^{next} .

The probability of selecting hypothesis h_i from H is:

% thực hiện tối đa n chuyển sang thế hệ tiếp theo

$$P(h_i) = \frac{\text{Fitness}(h_i)}{\sum_{j=1}^n \text{Fitness}(h_j)}$$

tỉ lệ thuận vs hàm fitness

GA(Fitness, θ , n , r_{co} , r_{mu})

...

Crossover.

Probabilistically select $(r_{co} \cdot n / 2)$ pairs of hypotheses from H , according to the probability computation $P(h_i)$ given above.

For each pair (h_i, h_j) , produce two offspring (i.e., children) by applying the crossover operator. Then, add all the offspring to H^{next} .

Mutation.

Select $(r_{mu} \cdot n)$ hypotheses of H^{next} , with uniform probability.

For each selected hypothesis, invert one randomly chosen bit (i.e., 0 to 1, or 1 to 0) in the hypothesis's representation.

Producing the **next generation**: $H \leftarrow H^{next}$

Evaluate the new population. For each $h \in H$: compute $\text{Fitness}(h)$

end while

return $\text{argmax}_{\{h \in H\}} \text{Fitness}(h)$

VD:

Bài toán: Tìm giá trị cực đại của hàm x^2 trong $10, 1, \dots, 315$

GA: - Biểu diễn dưới dạng chuỗi nhị phân. VD: $01101 \leftrightarrow 13$

- Kích thước quần thể: 4

- Toán tử lai ghép 1 điểm cắt, đột biến tại 1 điểm

- Lựa chọn kiểu Roulette wheel

- Khởi tạo ngẫu nhiên

11100

11001

11011

10100

Toán tử lựa chọn

String no.	Initial population	x Value	Fitness $f(x) = x^2$	$Prob_i$	Expected count	Actual count
1	0 1 1 0 1	13	169	0.14	0.58	1
2	1 1 0 0 0	24	576	0.49	1.97	2
3	0 1 0 0 0	8	64	0.06	0.22	0
4	1 0 0 1 1	19	361	0.31	1.23	1
Sum			1170	1.00	4.00	4
Average			293	0.25	1.00	1
Max			576	0.49	1.97	2

Lai ghép

String no.	Mating pool	Crossover point	Offspring after xover	x Value	Fitness $f(x) = x^2$
1	0 1 1 0 1	4	0 1 1 0 0	12	144
2	1 1 0 0 0	4	1 1 0 0 1	25	625
2	1 1 0 0 0	2	1 1 0 1 1	27	729
4	1 0 0 1 1	2	1 0 0 0 0	16	256
Sum					1754
Average					439
Max					729

Đột biến

String no.	Offspring after xover	Offspring after mutation	x Value	Fitness $f(x) = x^2$
1	0 1 1 0 0	1 1 1 0 0	28	784
2	1 1 0 0 1	1 1 0 0 1	25	625
2	1 1 0 1 1	1 1 0 1 1	27	729
4	1 0 0 0 0	1 0 1 0 0	20	400
Sum				2538
Average				634.5
Max				729

Ví dụ về tối ưu hoá hàm

- Bài toán: tìm giá trị cực đại của hàm: $f = x_1^2 - x_2$ với x_1 nguyên trong khoảng $[0,15]$ và x_2 nguyên trong khoảng $[0,31]$

Bước 1:

- Chọn mã hoá nhị phân $\{0,1\}$ với 4 gen cho x_1 , và 5 gen cho x_2
011010001 tương ứng với $x_1 = 0110 = 6$; $x_2 = 10001 = 17$
- Sử dụng sơ đồ chọn lọc tỷ lệ, toán tử lai ghép 1 điểm cắt, toán tử đột biến biến đổi 0 thành 1 và ngược lại, toán tử tái tạo không tinh hoa quần thể con P3 trở thành quần thể cho thế hệ tiếp theo.
- Fitness = Target - min Target trong quần thể + 1
- Chọn tham số $N = 4$, $P_c = 0.75$, $P_m = 0.01$, $G = 100$

- Bước 2:
- Khởi động quần thể đầu tiên ngẫu nhiên

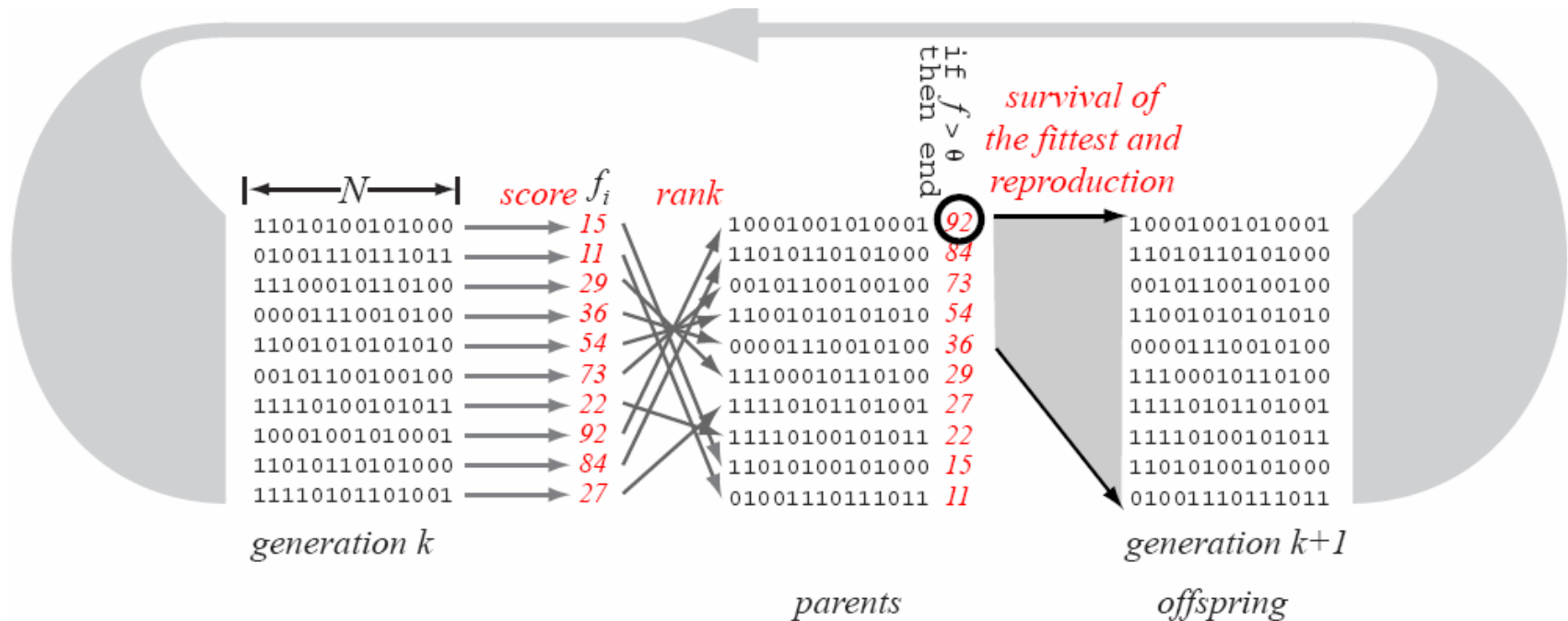
	Cá thể	x1	x2	giá trị hàm mục tiêu	giá trị độ phù hợp	Xác suất chọn lựa	Số bản copy trong P2	Giá trị trung bình
1	010011101	4	29	-13	1	0.0047	0	38.75
2	100001110	8	14	50	64	0.3033	1	
3	101010011	10	19	81	95	0.4502	2	
4	011101100	7	12	37	51	0.2417	1	

1010
0011
1110
1011

- Tiến hành lai ghép và đột biến tạo ra quần thể của thế hệ tiếp theo

Cá thể số	Bố mẹ	Vị trí lai ghép	con	con sau đột biến	x1	x2	Giá trị hàm mục tiêu	Giá trị t/bình
2	100001110	3	100010011	100010011	8	19	45	86.5
3	101010011		101001110	101001110	10	13	87	
3	101010011	7	101010000	111010000	14	16	180	
4	011101100		011101111	011101111	7	15	34	

Giải thuật di truyền: minh họa



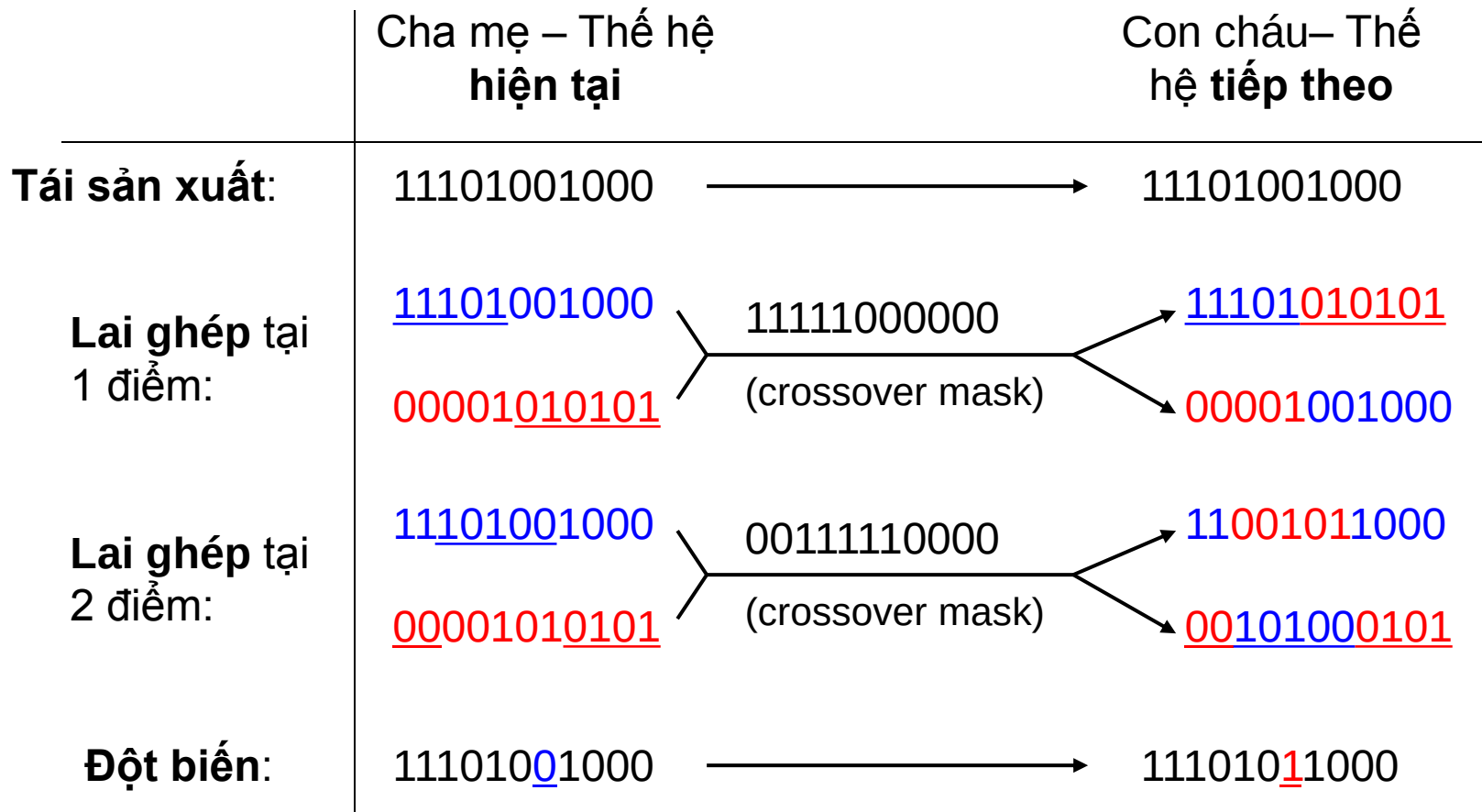
Để tạo ra lời giải tốt cho các thể hệ tiếp theo:

- Các cá thể cạnh tranh với nhau, cá thể có hàm fitness lớn hơn (cá thể thích nghi cao hơn) sẽ có n khả năng được sử dụng để tạo ra n hậu duệ lớn
 - Gen từ các cá thể tốt hơn kết hợp vs nhau $\rightarrow$ tạo ra hậu duệ tốt hơn
 - Từ cá thể tạo ra sự thay đổi gen của nó nhờ qtr' mutation
- [Duda et al., 2000]

Các toán tử di truyền

- 3 toán tử di truyền được sử dụng để sinh ra các cá thể con cháu (offspring) trong thế hệ tiếp theo
 - Nhưng chỉ có 2 toán tử lai ghép (*crossover*) và đột biến (*mutation*) tạo nên sự thay đổi
- **Tái sản xuất (Reproduction)**
 - Một giả thiết được giữ lại (không thay đổi)
- **Lai ghép (Crossover) để sinh ra 2 cá thể mới**
 - Ghép (“phối hợp”) của hai cá thể cha mẹ
 - Điểm lai ghép được chọn ngẫu nhiên (trên chiều dài của nhiễm sắc thể)
 - Phần đầu tiên của nhiễm sắc thể h_i được ghép với phần sau của nhiễm sắc thể h_j , và ngược lại, để sinh ra 2 nhiễm sắc thể mới
- **Đột biến (Mutation) để sinh ra 1 cá thể mới**
 - Chọn ngẫu nhiên một bit của nhiễm sắc thể, và đổi giá trị ($0 \rightarrow 1$ / $1 \rightarrow 0$)
 - Chỉ tạo nên một thay đổi nhỏ và ngẫu nhiên đối với một cá thể cha mẹ!

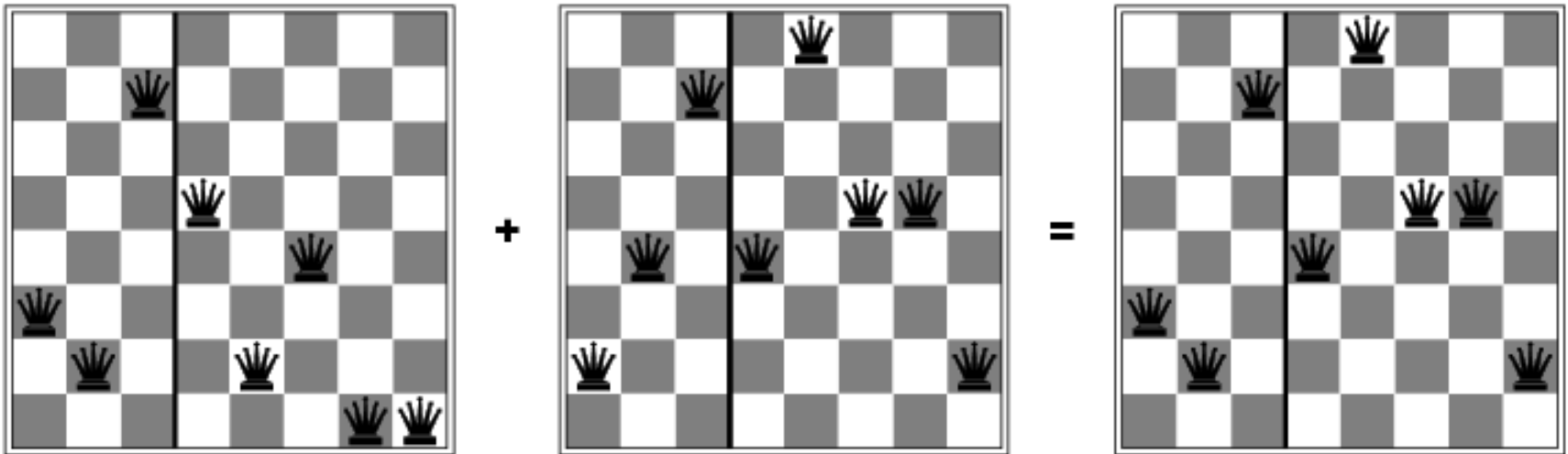
Các toán tử di truyền: ví dụ



[Mitchell, 1997]

Toán tử lai ghép: ví dụ

Bài toán bố trí 8 quân hậu trên bàn cờ - Toán tử lai ghép (crossover)



Tìm kiếm có đối thủ

- Các thủ tục tìm kiếm sâu dần (IDS) và tìm kiếm A^* hữu dụng đối với các bài toán (tìm kiếm) liên quan đến một tác tử
- Thủ tục tìm kiếm cho các bài toán liên quan đến 2 tác tử có mục tiêu đối nghịch nhau (xung đột với nhau)?
 - Tìm kiếm có đối thủ (Adversarial search)
- Phương pháp tìm kiếm có đối thủ được áp dụng phổ biến trong các trò chơi (games)

Các vấn đề của tìm kiếm trong trò chơi

- Khó dự đoán trước được phản ứng của đối thủ
 - Cần xác định (xét) một nước đi phù hợp đối với mỗi phản ứng (nước đi) có thể của đối thủ
- Giới hạn về thời gian (trò chơi có tính giờ)
 - Thường khó (hoặc không thể) tìm được giải pháp tối ưu → Xấp xỉ
- Tìm kiếm có đối thủ đòi hỏi tính hiệu quả (giữa chất lượng của nước đi và chi phí thời gian)
Đây là một yêu cầu khó khăn
- Nguyên tắc trong nhiều trò chơi đối kháng
 - Một người chơi thắng = Người chơi kia thua
 - Mức độ (tổng điểm) thắng của một người chơi = Mức độ (tổng điểm) thua của người chơi kia

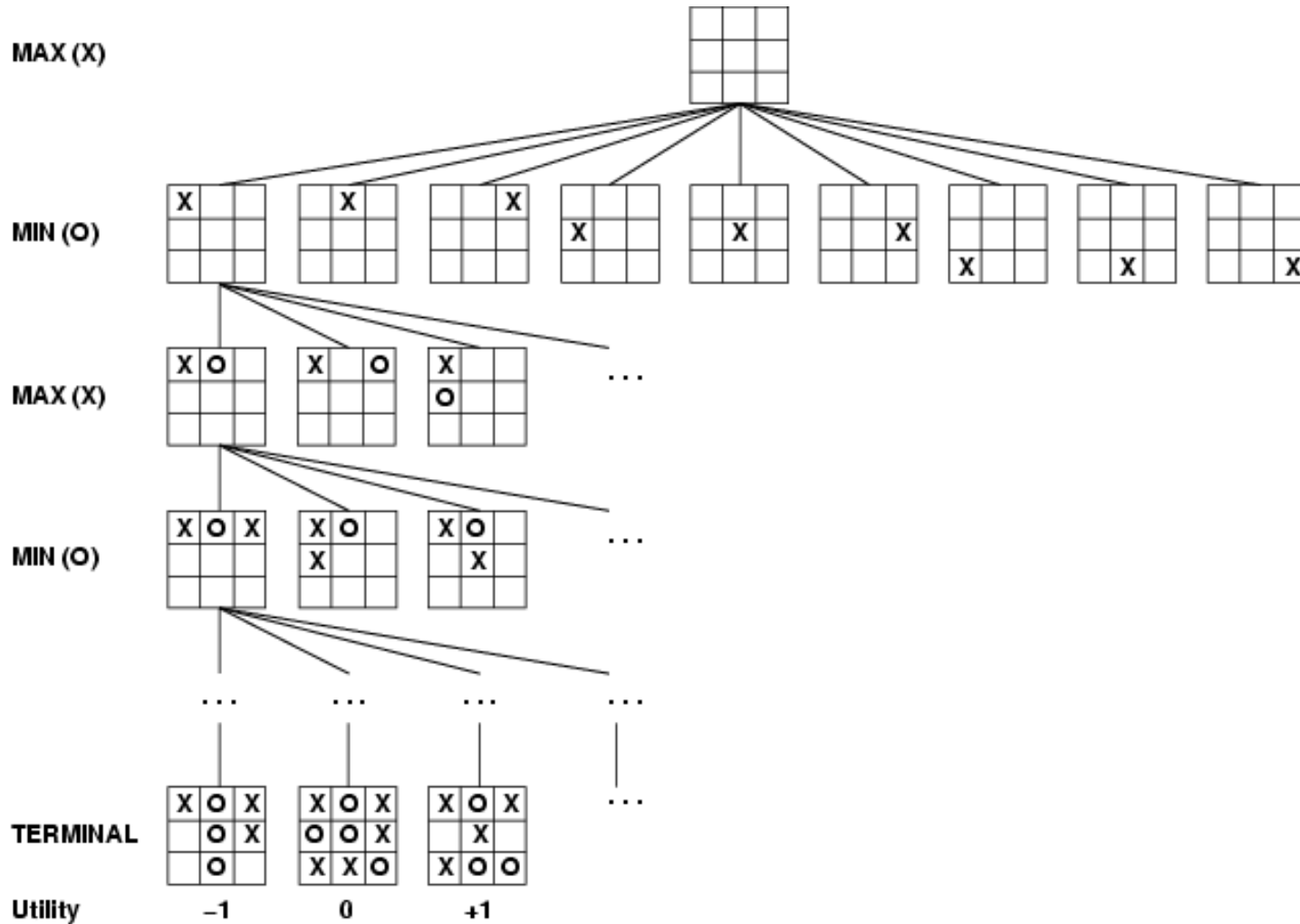
Trò chơi cờ ca-rô

- Trò chơi cờ ca-rô là một ví dụ phổ biến trong AI để minh họa về tìm kiếm có đối thủ
 - Vd: <http://www.ourvirtualmall.com/tictac.htm>
- Là trò chơi đối kháng giữa 2 người (gọi là MAX và MIN)
 - Thay phiên nhau đi các nước (moves)
 - Kết thúc trò chơi: Người thắng được thưởng (điểm), người thua bị phạt (điểm)

Biểu diễn bài toán trò chơi đối kháng

- Trò chơi bao gồm các thông tin
 - Trạng thái bắt đầu (Initial state): Trạng thái của trò chơi + Người chơi nào được đi nước đầu tiên
 - Hàm chuyển trạng thái (Sucessor function): Trả về thông tin gồm (nước đi, trạng thái)
 - Tất cả các nước đi hợp lệ từ trạng thái hiện tại
 - Trạng thái mới (là trạng thái chuyển đến sau nước đi)
 - Kiểm tra kết thúc trò chơi (Terminal test)
 - Hàm tiện ích (Utility function) để đánh giá các trạng thái
- Trạng thái bắt đầu + Các nước đi hợp lệ = Cây biểu diễn trò chơi (Game tree)

Cây biểu diễn trò chơi cờ ca-rô

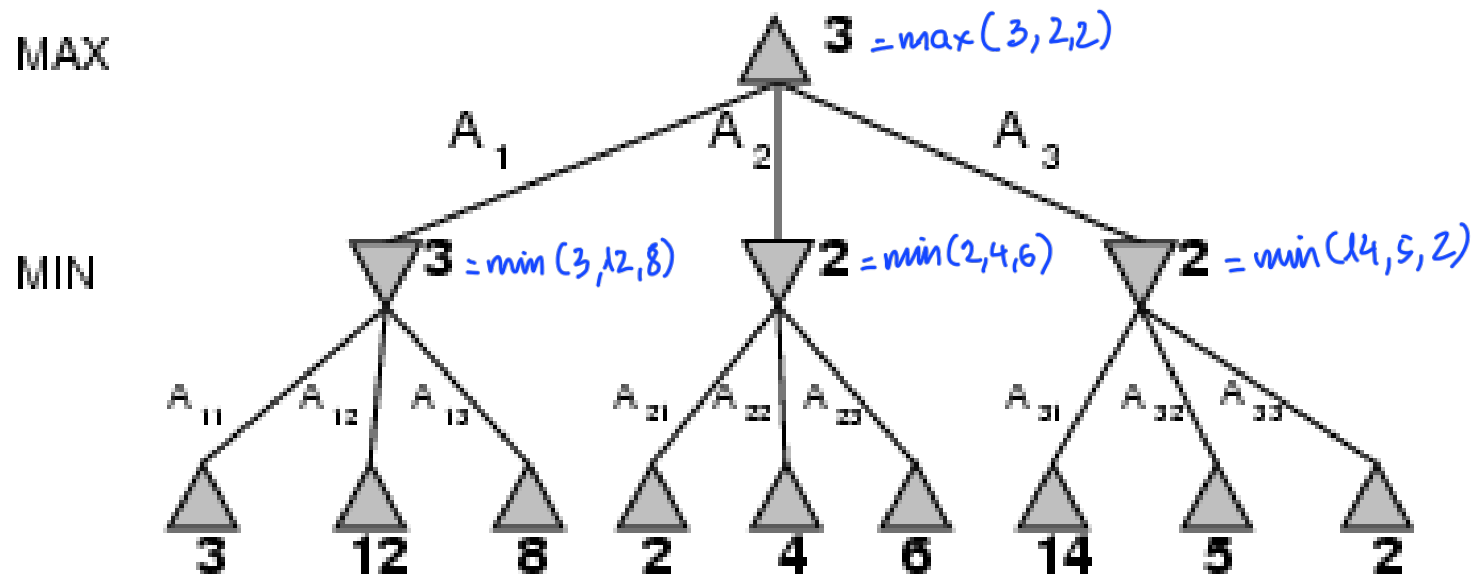


Các chiến lược tối ưu

- Một chiến lược tối ưu là một chuỗi các nước đi giúp đưa đến trạng thái đích mong muốn (vd: chiến thắng)
- Chiến lược của MAX bị ảnh hưởng (phụ thuộc) vào các nước đi của MIN – và ngược lại
- MAX cần chọn một chiến lược giúp cực đại hóa giá trị hàm mục tiêu – với giả sử là MIN đi các nước đi tối ưu
 - MIN cần chọn một chiến lược giúp cực tiểu hóa giá trị hàm mục tiêu
- Chiến lược này được xác định bằng việc xét giá trị MINIMAX đối với mỗi nút trong cây biểu diễn trò chơi
 - Chiến lược tối ưu đối với các trò chơi có không gian trạng thái xác định (deterministic states)

Giá trị MINIMAX

- MAX chọn nước đi ứng với giá trị MINIMAX cực đại (để đạt được giá trị cực đại của hàm mục tiêu)
- Ngược lại, MIN chọn nước đi ứng với giá trị MINIMAX cực tiểu



Giải thuật MINIMAX

function MINIMAX-DECISION(*state*) *returns an action*

$v \leftarrow \text{MAX-VALUE}(\textit{state})$

return the *action* in **SUCCESSORS**(*state*) with value *v*

function MAX-VALUE(*state*) *returns a utility value*

if **TERMINAL-TEST**(*state*) **then return** **UTILITY**(*state*)

$v \leftarrow -\infty$

for *a, s* in **SUCCESSORS**(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$

return *v*

function MIN-VALUE(*state*) *returns a utility value*

if **TERMINAL-TEST**(*state*) **then return** **UTILITY**(*state*)

$v \leftarrow \infty$

for *a, s* in **SUCCESSORS**(*state*) **do**

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$

return *v*

```
function minimax (node, depth, maximizingPlayer)
  if node is EndNode or depth = 0
    return value (node)
  if maximizingPlayer
     $M_x = -\infty$ 
    foreach child of node
       $M_x = \max(M_x, \text{minimax}(\text{child}, \text{depth}-1, \text{False}))$ 
    return  $M_x$ 
  else:
     $M_n = \infty$ 
    foreach child of node
       $M_n = \min(M_n, \text{minimax}(\text{child}, \text{depth}-1, \text{True}))$ 
    return  $M_n$ 
```

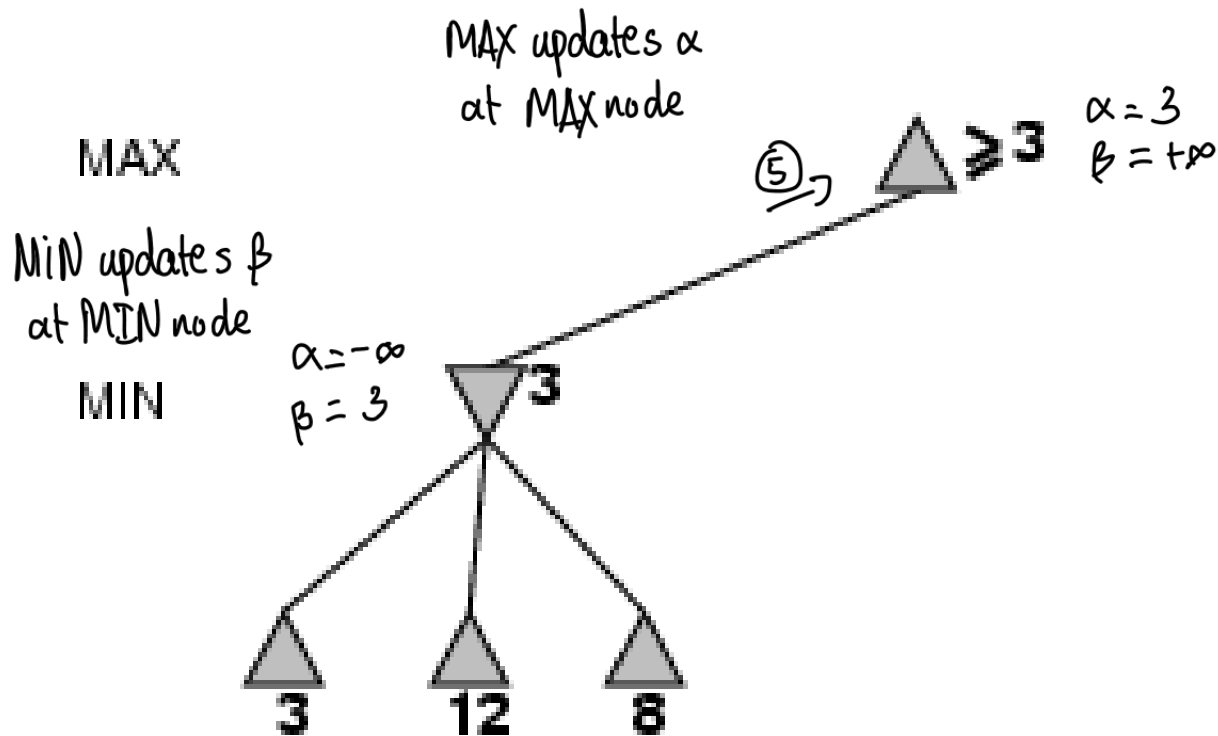
Giải thuật MINIMAX: các đặc điểm

- Tính hoàn chỉnh
 - Có (nếu cây biểu diễn trò chơi là hữu hạn)
- Tính tối ưu
 - Có (đối với một đối thủ luôn chọn nước đi tối ưu)
- Độ phức tạp về thời gian
 - $O(b^m)$
- Độ phức tạp về bộ nhớ
 - $O(bm)$ (khám phá theo chiến lược tìm kiếm theo chiều sâu)
- Đối với trò chơi cờ vua, hệ số phân nhánh $b \approx 35$ và hệ số mức độ sâu của cây biểu diễn $m \approx 100$
 - Chi phí quá cao – Không thể tìm kiếm chính xác nước đi tối ưu

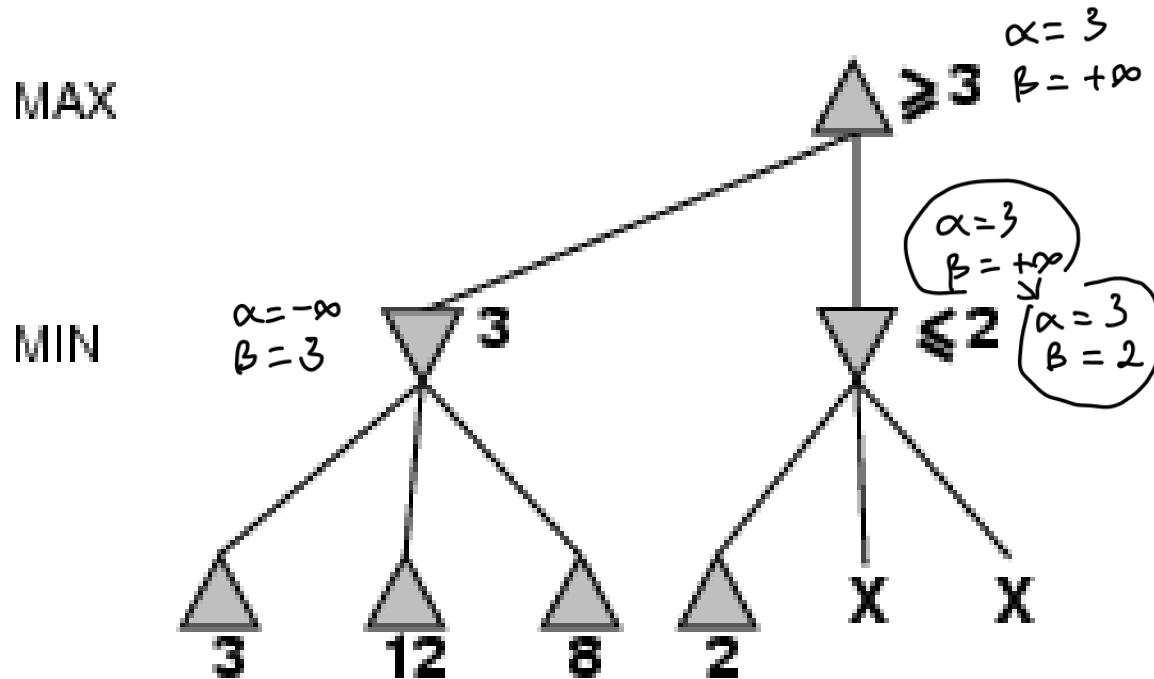
Cắt tỉa tìm kiếm

- Vấn đề: Giải thuật tìm kiếm MINIMAX vấp phải vấn đề bùng nổ (mức hàm mũ) các khả năng nước đi cần phải xét → không phù hợp với nhiều bài toán trò chơi thực tế
- Chúng ta có thể cắt tỉa (bỏ đi – không xét đến) một số nhánh tìm kiếm trong cây biểu diễn trò chơi
- Phương pháp cắt tỉa α - β (Alpha-beta pruning)
 - Ý tưởng: Nếu một nhánh tìm kiếm nào đó không thể cải thiện đối với giá trị (hàm tiện ích) mà chúng ta đã có, thì không cần xét đến nhánh tìm kiếm đó nữa!
 - Việc cắt tỉa các nhánh tìm kiếm (“tôi”) không ảnh hưởng đến kết quả cuối cùng

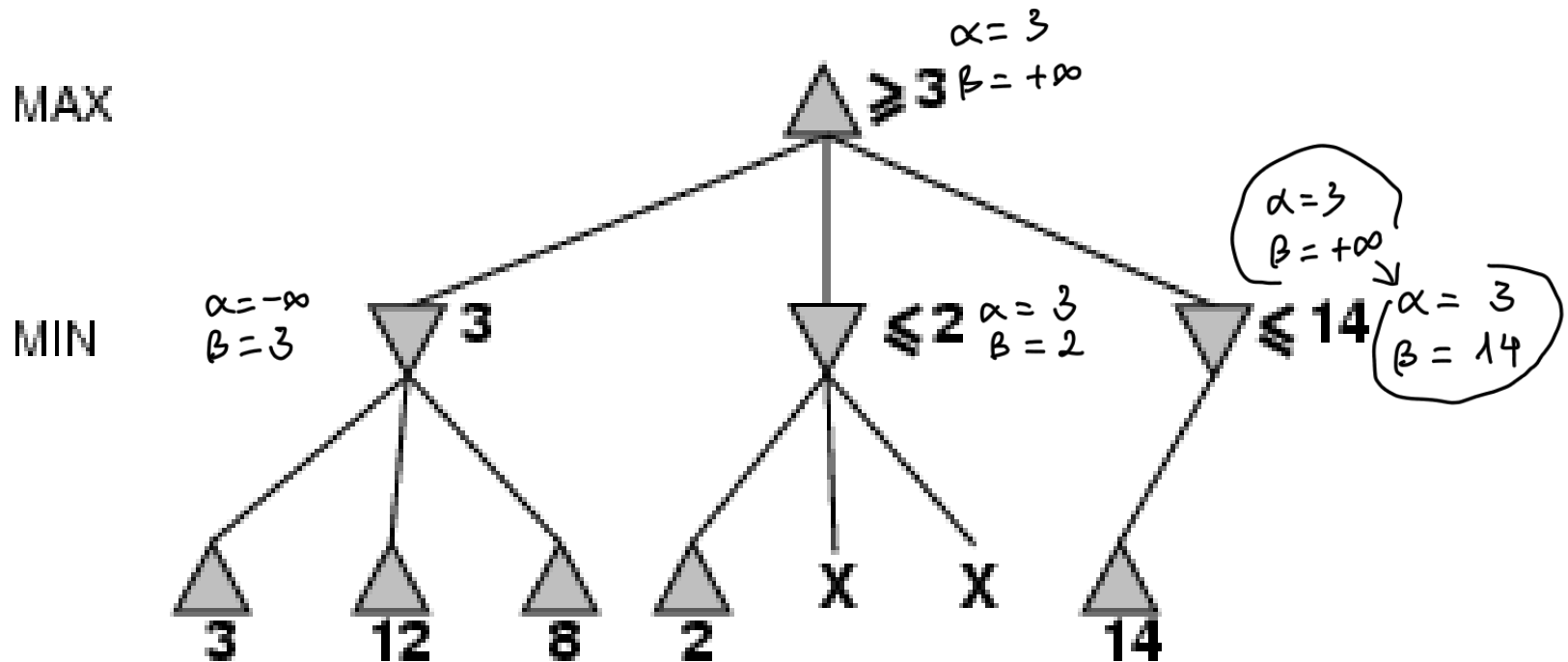
Cắt tỉa α - β – Ví dụ (1)



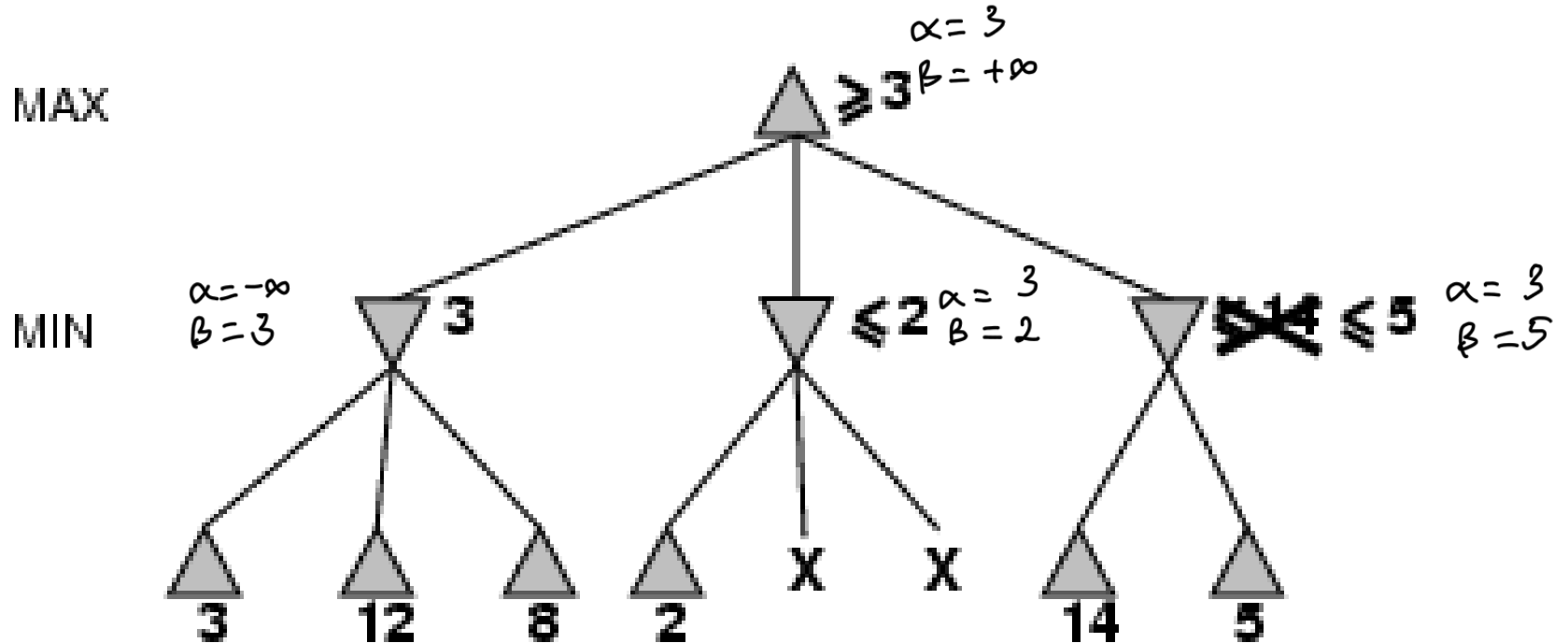
Cắt tỉa α - β – Ví dụ (2)



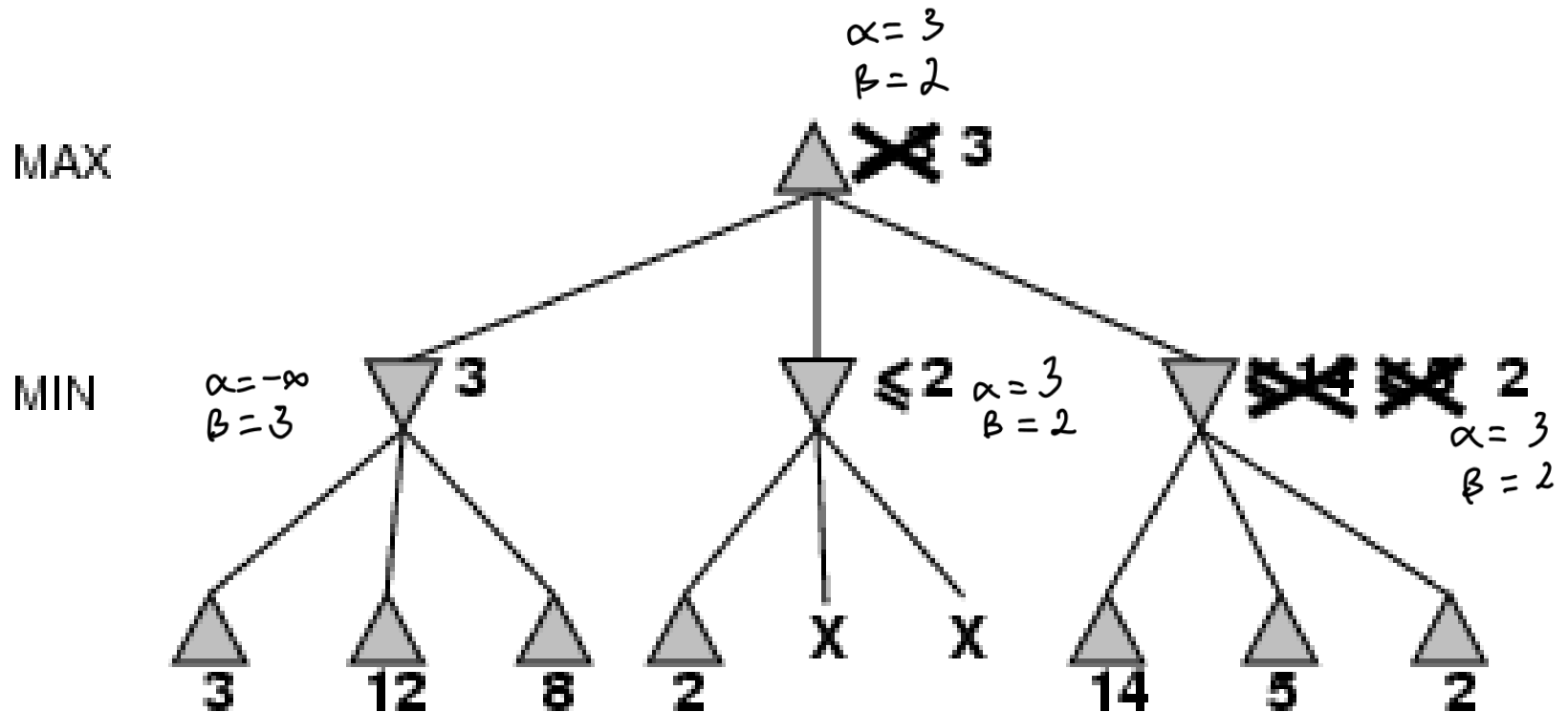
Cắt tỉa α - β – Ví dụ (3)



Cắt tỉa α - β – Ví dụ (4)

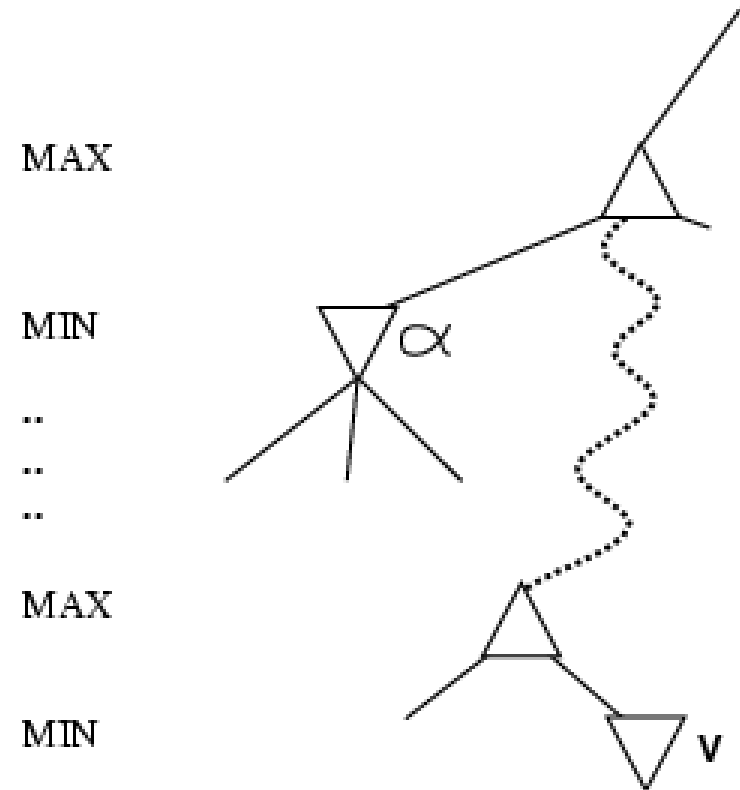


Cắt tỉa α - β – Ví dụ (5)



Tại sao được gọi là cắt tỉa α - β ?

- α là giá trị của nước đi tốt nhất đối với MAX (giá trị tối đa) tính đến hiện tại đối với nhánh tìm kiếm
- Nếu v là giá trị tồi hơn α , MAX sẽ bỏ qua nước đi ứng với v
 - Cắt tỉa nhánh ứng với v
- β được định nghĩa tương tự đối với MIN



Giải thuật cắt tỉa α - β (1)

function ALPHA-BETA-SEARCH(*state*) *returns an action*

inputs: *state*, current state in game

$v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$

return the *action* in **SUCCESSORS**(*state*) with value v

function MAX-VALUE(*state*, α , β) *returns a utility value*

inputs: *state*, current state in game

α , the value of the best alternative for MAX along the path to *state*

β , the value of the best alternative for MIN along the path to *state*

if **TERMINAL-TEST**(*state*) **then return** **UTILITY**(*state*)

$v \leftarrow -\infty$

for a, s in **SUCCESSORS**(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$

if $v \geq \beta$ **then return** v

$\alpha \leftarrow \text{MAX}(\alpha, v)$

return v

Giải thuật cắt tỉa α - β (2)

```
function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  inputs: state, current state in game
            $\alpha$ , the value of the best alternative for MAX along the path to state
            $\beta$ , the value of the best alternative for MIN along the path to state

  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow +\infty$ 
  for  $a, s$  in SUCCESSORS(state) do
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s, \alpha, \beta))$ 
    if  $v \leq \alpha$  then return  $v$ 
     $\beta \leftarrow \text{MIN}(\beta, v)$ 
  return  $v$ 
```

```
function minimax(Node, depth, maximizing Player)
    return alphabeta(Node, depth,  $-\infty$ ,  $\infty$ , maximizing Player)
```

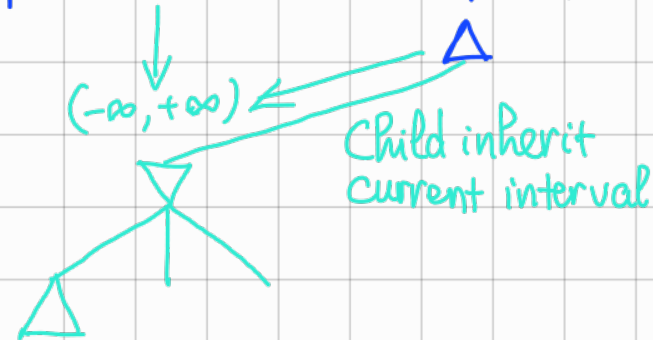
```
function alphabeta(Node, depth, a, b, maximizing Player)
    if node is EndNode or depth = 0
        return value(Node)
    if maximizing Player
        foreach child of node
            a = max(a, alphabeta(child, depth-1, a, b, False))
            if a >= b break;
        return a
    else
        foreach child of node
            b = min(b, alphabeta(child, depth-1, a, b, True))
            if a >= b break;
        return b
```

"Possible Interval" view: (min, max)

- ② Do DFS until first leaf

MAX

MIN



- ④ See remaining leaves and outcome is known

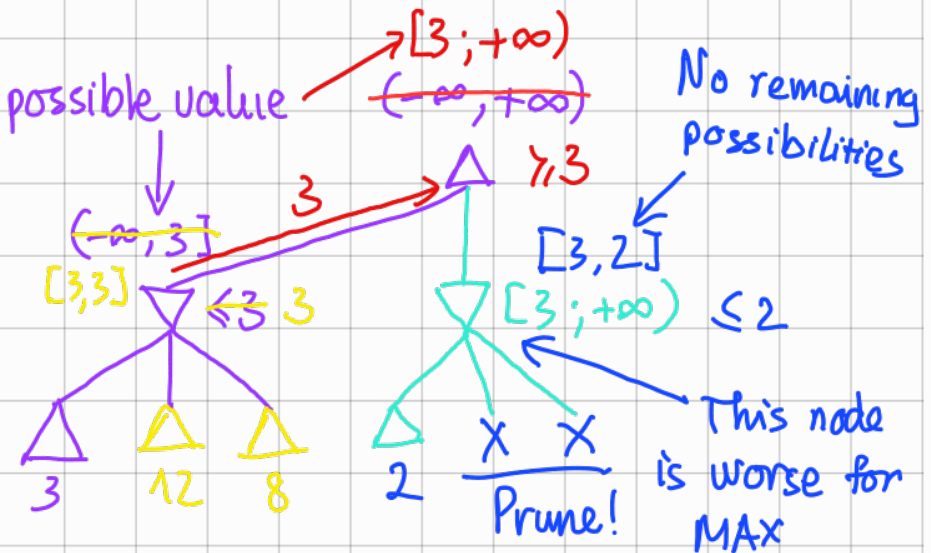
- ⑤ Pass outcome to caller, and update caller

Then $(2) \rightarrow (3) \rightarrow \dots$

Range of possible value

MAX

MIN

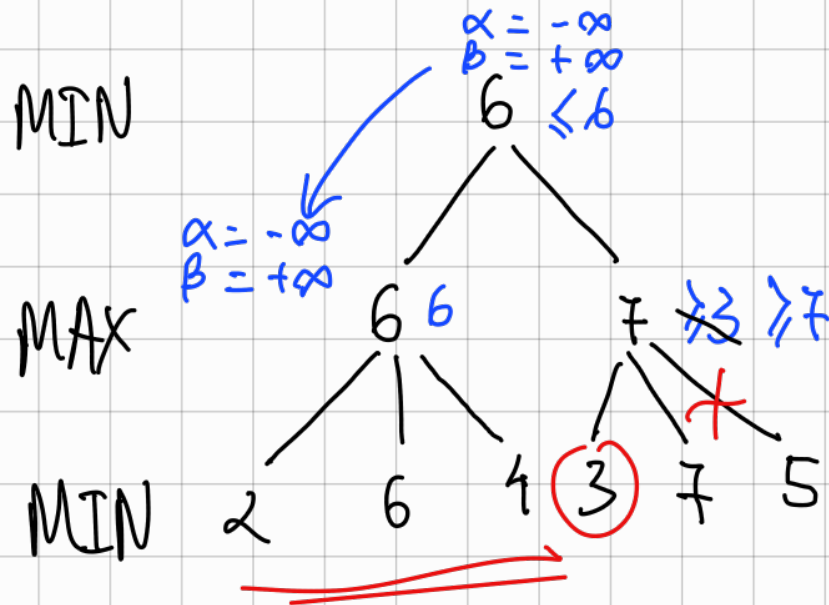


α : lựa chọn có gtn tìm thấy trg bất kỳ đ' lựa chọn trên đường đi của MAX (bđầu: $\alpha = -\infty$)
 β : " " " gtnn " " " " " " " " " " MIN (" : $\beta = +\infty$)

prune when $\alpha \geq \beta$

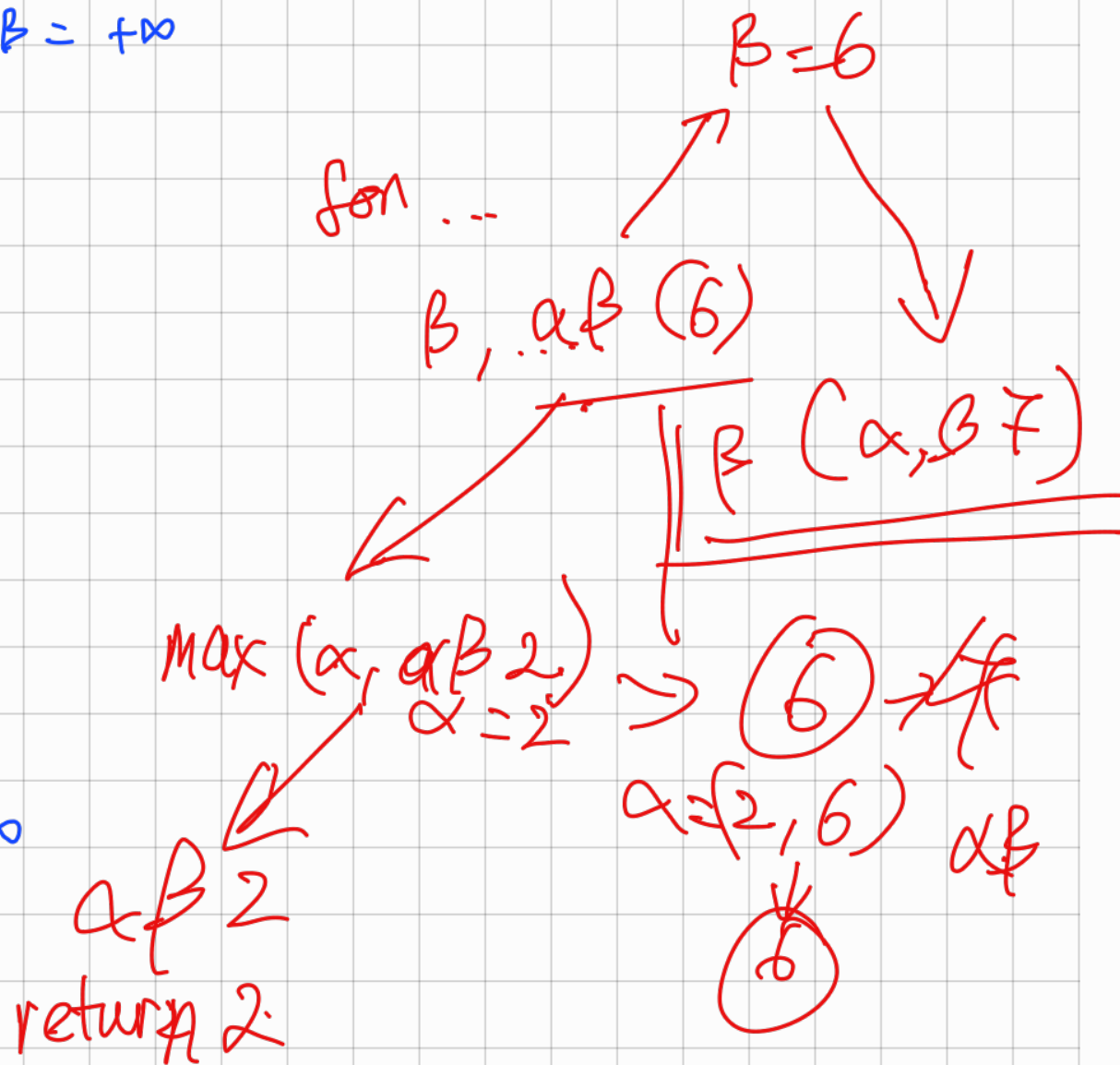
$$\alpha = -\infty$$

$$\beta = +\infty$$



$$\alpha \uparrow, \beta$$

$$\alpha = \max(\alpha, \alpha\beta(7)) = 7$$



$$\min(\beta, \alpha\beta(7))$$

$$\alpha = \max(\alpha, \alpha\beta(3)) = 6$$

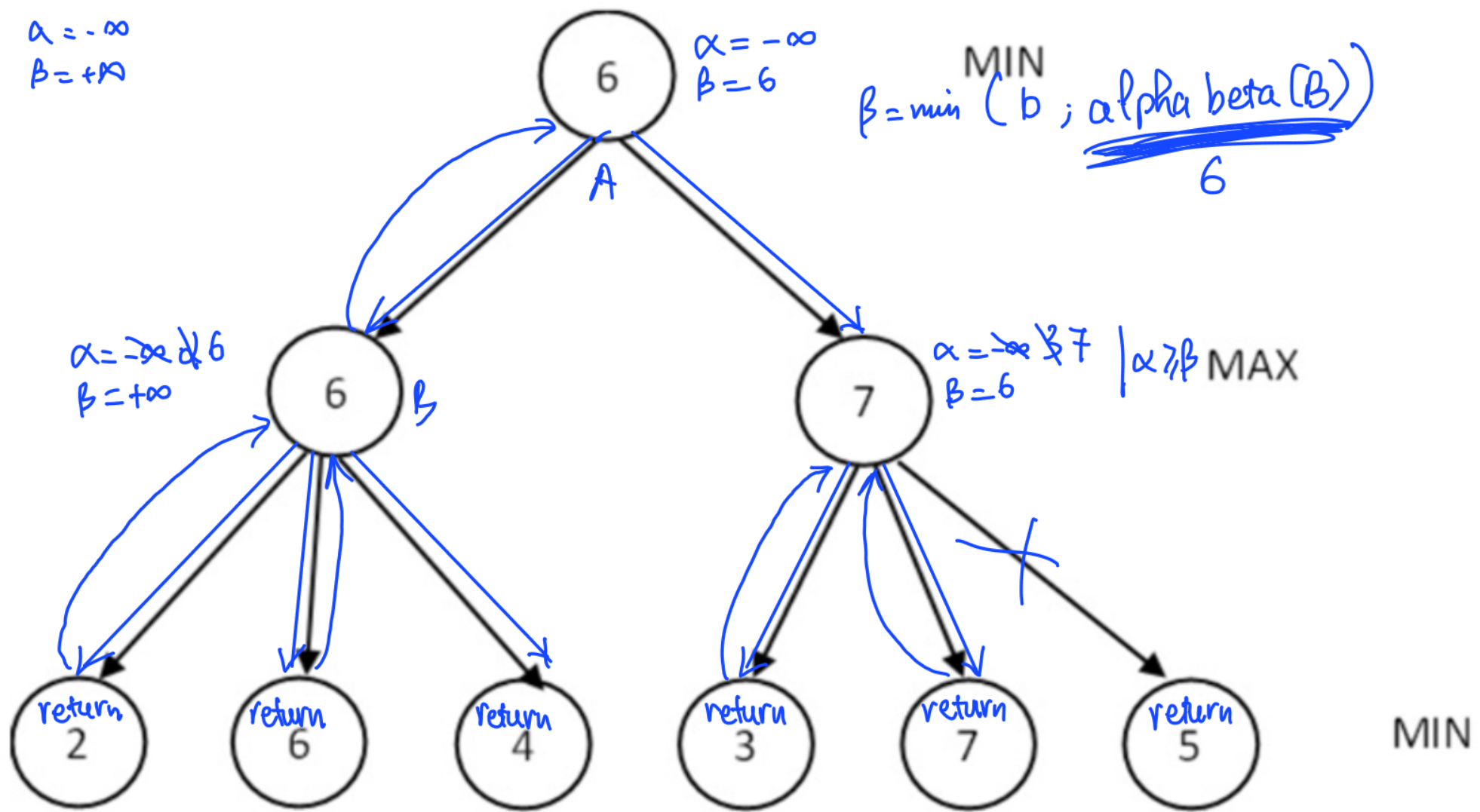
$\alpha = -\infty$
 $\beta = +\infty$

$\alpha = -\infty$
 $\beta = 6$

MIN
 $\beta = \min(b; \text{alpha beta}(B))$
6

$\alpha = -\infty$
 $\beta = +\infty$

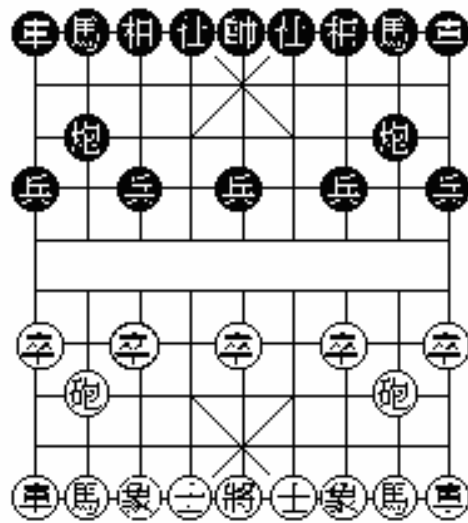
$\alpha = -\infty$
 $\beta = 6$
 $\alpha \geq \beta$ MAX



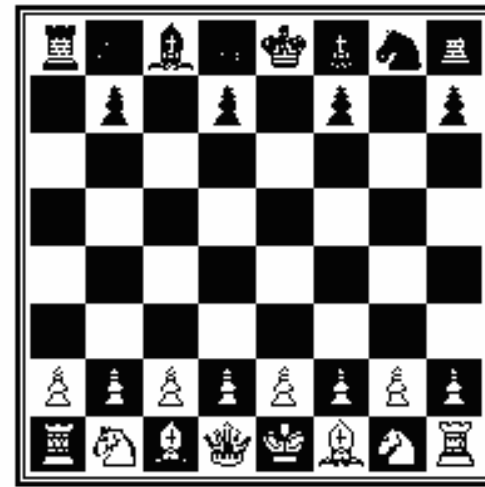
Cắt tỉa α - β

- Đối với các trò chơi có không gian trạng thái lớn, thì phương pháp cắt tỉa α - β vẫn không phù hợp
 - Không gian tìm kiếm (kết hợp cắt tỉa) vẫn lớn
- Có thể hạn chế không gian tìm kiếm bằng cách sử dụng các tri thức cụ thể của bài toán
 - Tri thức để cho phép đánh giá mỗi trạng thái của trò chơi
 - Tri thức bổ sung (heuristic) này đóng vai trò tương tự như là hàm ước lượng $h(n)$ trong giải thuật tìm kiếm A^*

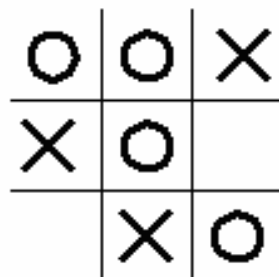
Một số trò chơi đối kháng (minimax)



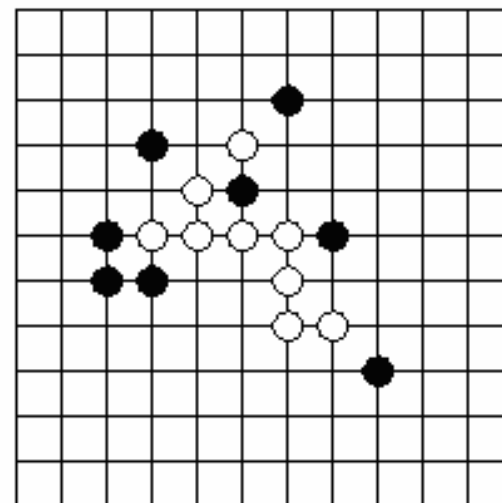
a) Cờ Tướng



b) Cờ Vua (cờ Quốc Tế)



c) Tictactoe



d) Go-moku (cờ caro)